HANFIS: A Hierarchical Adaptive Neuro-Fuzzy Inference System for Evidence-Based Dynamic Cybersecurity Risk Scoring in Cloud-Native Kubernetes Deployments

Mohammad Hafiz Hersyah[1*], Derisma Derisma[2] and Budi Rahmadya[1]

[1*]Computer Engineering Department, Andalas University, Kampus Limau Manis Street, Padang, 25162, West Sumatera, Indonesia.
[2]Informatics Department, Andalas University, Kampus Limau Manis Street, Padang, 25162, West Sumatera, Indonesia.

*Corresponding author(s). E-mail(s): mohammadhafizhersyah02@it.unand.ac.id;
Contributing authors: derisma@it.unand.ac.id; budi-r@it.unand.ac.id;

Abstract

Static, expert-weighted fuzzy risk scores go stale between elicitations, and a flat Adaptive Neuro-Fuzzy Inference System (ANFIS) over seven risk inputs needs 128 rules and 1,052 parameters. We built a hierarchical ANFIS (HANFIS) that separates vulnerability and environmental reasoning before an integration tier adds threat intelligence (34 rules, 176 parameters) and tested it against simple baselines. The target is exploitability exposure from sealed weekly Exploit Prediction Scoring System (EPSS) and Known Exploited Vulnerabilities (KEV) snapshots for 120 container images (6,357 labelled asset-weeks), evaluated with grouped, embargoed splits. A 4-week persistence baseline lowers mean squared error relative to climatology by 97%; a single Common Vulnerability Scoring System (CVSS) score, by 22%. Under a protocol frozen before training, HANFIS reduces error by 96.7% against CVSS, 69.6% against a ridge model on all

seven inputs and 66.5% against a parameter-matched flat ANFIS (95% bootstrap intervals excluding zero; the last margin comes largely from one fold), and it is not distinguishable from persistence. Post hoc analyses narrow these gains. Exact Shapley attribution puts 78.9% on exploitability and the KEV indicator, both label ingredients; without them HANFIS has no skill over CVSS (skill $\mathbf{-0.14}$, interval $\mathbf{-0.34}$ to $\mathbf{0.05}$). A four-parameter formula of those two inputs, $\mathbf{100\,\max(x_2, 0.1\,x_7)}$, beats HANFIS at four weeks (skill $\mathbf{-0.53}$, interval $\mathbf{-1.44}$ to $\mathbf{-0.21}$). In a pre-registered forward evaluation the formula beat persistence and HANFIS did not. On this target HANFIS offers a printable rule base and no gain in accuracy, and the data cannot show whether environmental context helps.

Keywords: Adaptive Neuro-Fuzzy Inference System, Exploit prediction, Security Risk Assessment, Container Orchestration, Cloud-Native Security

1 Introduction

Cloud-native computing has changed both the scale and the temporal behaviour of enterprise attack surfaces. Applications are decomposed into loosely coupled microservices on container orchestration, and Kubernetes Pods are deliberately ephemeral: they can be created, replaced, rescheduled, or terminated as conditions change Chandramouli (2019); The Kubernetes Authors (2026). This shortens the useful life of security assessments derived from periodic observations, because a risk value calculated before a deployment, configuration change, or new exploitation signal may no longer describe the workload when a decision is needed. Zero Trust Architecture (ZTA) reinforces the problem by requiring access decisions to consider the current identity, resource, and operational context Rose et al. (2020); Chandramouli and Butcher (2023). Cloud-native risk assessment therefore has to follow changes in vulnerability status, workload exposure, authentication posture, asset importance, and external threat activity.

Our previous Fuzzyfortify framework Hafiz Hersyah et al. (2025) established an interpretable foundation for cybersecurity risk assessment by using fuzzy multi-criteria

reasoning to represent expert knowledge and linguistic uncertainty, which remains useful when observations are incomplete and priorities cannot be expressed through crisp thresholds. Its principal limitation is temporal rather than representational: expert-derived weights and rules stay fixed until they are elicited again, so a score from a static configuration may remain unchanged even when telemetry indicates active exploitation, loss of multifactor authentication protection, increased Kubernetes exposure, or delayed remediation. What is missing is a reproducible mechanism that preserves expert interpretability while adapting the input–output relationship from observed data and recalculating risk from the current system state.

An Adaptive Neuro-Fuzzy Inference System (ANFIS) is a suitable basis for this gap because it combines human-readable fuzzy rules with data-driven adjustment of membership and consequent parameters Jang (1993). A flat ANFIS, however, becomes difficult to train and inspect as its input dimension grows: under exhaustive grid partitioning, n inputs with m membership functions each may require m^n rules, and for the seven inputs considered here two membership functions per input already require $2^7 = 128$ rules before any pruning. Such growth increases the number of adaptive parameters, the size of the least-squares problem, memory consumption, and inference operations.

This study builds and tests a domain-informed Hierarchical Adaptive Neuro-Fuzzy Inference System (HANFIS). The first tier contains two parallel subsystems. The vulnerability subsystem combines the CVSS Base Score, Exploitability, and Patch Risk to produce a Vulnerability Risk Index. The environmental subsystem combines Multi-Factor Authentication (MFA) Risk, Kubernetes Exposure, and Asset Criticality to produce an Environmental Risk Index. A second-tier integration subsystem combines these two intermediate indices with a dynamic Threat Intelligence input to produce the final Dynamic Risk Score. This organization is consistent with established risk-assessment logic, in which vulnerability and threat information are interpreted within

an organizational context Joint Task Force Transformation Initiative (2012). It also prevents threat information from being counted twice by separating the CVSS Base Score from the dynamic threat signal, in accordance with the distinction among Base, Threat, and Environmental metric groups in CVSS v4.0 FIRST.org, Inc. (2023). Threat Intelligence can be operationalized through time-varying evidence such as EPSS probability, known exploitation status, exploitation recency, and source confidence; EPSS, in particular, estimates the probability that a published vulnerability will be exploited in the following 30 days Jacobs et al. (2021).

The hierarchical decomposition also changes the rule count directly. If every input uses m membership functions, the exhaustive rule count is reduced from

$$R_{\text{flat}} = m^7 \tag{1}$$

to

$$R_{\text{HANFIS}} = 2m^3 + m_I^2\, m, \tag{2}$$

where m is the number of membership functions on each of the seven raw inputs and m_I is the number of membership functions on the two intermediate variables v and e inside the integration tier (Section 3.2). With the frozen configuration $m = 2$, $m_I = 3$ (amendment A-02), the proposed structure requires 34 candidate rules rather than 128, a 73.4% reduction; the corresponding parameter count falls from 1,052 to 176, an 83.3% reduction (Table 1). Hierarchical fuzzy systems have been shown to mitigate dimensionality-driven rule growth while retaining nonlinear approximation capability Wang (1998); Ojha et al. (2019). Nevertheless, a lower theoretical rule count does not by itself establish superiority. Hierarchical compression may discard cross-subsystem interactions, and the sequential integration tier introduces an additional inference stage. The proposed architecture must therefore be compared against a flat grid-partitioned ANFIS and a rule-reduced or clustering-based flat ANFIS, together

with simple baselines, using identical observations, data partitions, and evaluation conditions. Fuzzyfortify is not compared, because its native inputs are not produced by the asset-week data (Section 4.3).

Edge deployment of HANFIS is outside the claims of this paper. Its computational cost is compared with flat ANFIS variants on one machine (rule count, trainable parameters, training time, peak memory, throughput, and single-row inference latency; Section 4.15), and edge-platform measurements are left to future work.

A policy-translation layer that maps validated risk states to bounded responses is designed but not evaluated here, and it is outside the claims of this paper (Section 6).

The aim of this study is to find out whether a domain-informed hierarchical decomposition earns its place in cloud-native vulnerability risk scoring. We build a HANFIS with a reduced rule base and intermediate variables designed to be interpretable, test how faithful its rule-level explanations are (Section 4.6), and ask whether it preserves or improves estimation error relative to a flat ANFIS and, more demandingly, relative to two baselines that need no neuro-fuzzy training: a 4-week persistence forecast and a formula of two inputs. On the realised exploitability-exposure target the answer is mostly negative. The label is dominated by its own recent history, HANFIS is not distinguishable from persistence, and a four-parameter formula beats it in a pre-registered forward window. The paper reports that result together with the frozen protocol, the post hoc analyses that explain it, and the limits of what the data can show. Policy support and edge execution are designed but left to future work.

The contributions of this study are:

C1. A domain-informed HANFIS architecture, proposed as a design, that separates vulnerability and environmental reasoning before integrating their outputs with threat intelligence to produce a Dynamic Risk Score recalculated weekly from sealed EPSS and KEV snapshots. Unlike the speaker-verification HANFIS of Soozanchi-K. et al.

(2010), its three tiers follow a vulnerability, environment and threat decomposition, and the evaluation does not show that the environmental panel contributes (Sections 4.7 and 4.8).

C2. A hierarchical decomposition that changes exhaustive rule growth from m^7 to $2m^3 + m_I^2 m$ (the arithmetic of hierarchical fuzzy systems Raju et al. (1991); Wang (1998)), accompanied by an unregistered measurement of parameter count, training time, peak memory, throughput and single-row inference latency against flat ANFIS variants (Section 4.15). It lowers parameters and peak training memory and raises batch throughput relative to the full flat model, but not training time or single-row latency.

C3. An operational definition of seven risk inputs (CVSS Base severity, exploitability, patch condition, MFA weakness, Kubernetes exposure, asset criticality, and time-varying threat evidence, without duplicating CVSS threat metrics) and of a realised 30-day exploitability-exposure target built from sealed EPSS and KEV snapshots. The target has no impact term, so it measures exploitability exposure, not full cybersecurity risk.

C4. A reproducible evaluation against persistence, single-CVSS, ridge and flat ANFIS baselines (full and parameter-matched), using predictive and computational metrics under common grouped, embargoed temporal partitions. Post hoc information-minimal and formula baselines, an ablation of the label-ingredient inputs and a horizon sweep complement it (Sections 4.8 and 4.11). A pre-registered forward evaluation on six weeks after the protocol freeze tests the formula and HANFIS against persistence (Section 4.16). A Fuzzyfortify comparison is not claimed (Section 4.3).

C5. A pre-registered negative result for the hierarchical model on this target. Persistence alone removes 97% of the error relative to climatology, so a model trained on a label of this kind has to be judged against it. In the forward window HANFIS was not better than persistence and was worse than a four-parameter formula (Section 4.16),

and its skill rests on two inputs that are also ingredients of the label (Sections 4.7 and 4.8). The result concerns one architecture, one target and one six-week window, and it is not a verdict on neuro-fuzzy models in general.

The empirical analysis is guided by the following hypotheses:

H1a: Improvement over static expert scoring. HANFIS will produce significantly lower risk-estimation error than a single CVSS score, the current-practice baseline.

H1b: Improvement over naive persistence. HANFIS will reduce mean squared error relative to a causal 4-week persistence forecast by at least a smallest-effect-size-of-interest of 5%, with the lower bound of its 95% confidence interval above zero. Persistence, not a single CVSS score, is the empirically strongest baseline observed on the pilot dataset (Section 4.3) and is therefore the bar HANFIS must clear, not an optional comparator.

H1c: Value of hierarchical structure over a linear model. HANFIS will be non-inferior to a ridge-regularised linear model on the same seven inputs, with superiority reported when the confidence interval excludes zero.

H2: Computational efficiency. For an equivalent input set and membership-function configuration, HANFIS will require significantly fewer fuzzy rules and adaptive parameters and will reduce training time, peak memory, and inference latency relative to flat grid-partitioned ANFIS.

H3: Performance preservation. The predictive performance of HANFIS will be non-inferior to a parameter-matched flat ANFIS (FlatANFISMatched, ±5% parameters) within a prespecified margin, despite the reduced rule-base complexity.

H4: Threat responsiveness. Including time-varying Threat Intelligence will improve the prioritization of actively exploited or high-probability vulnerabilities compared with an otherwise identical HANFIS without this input.

H5 (edge execution), originally framed as a fifth hypothesis, is demoted to future work: the Jetson Orin Nano node in the present testbed is a telemetry client only, and HANFIS inference runs cluster-side, so no edge-execution timing evidence exists to test it against. H1a–H1c and H2–H3 are pre-registered against the frozen protocol, seed list, and data split described in Section 4.2; results are reported in Section 4.3 (Table 4). In the frozen protocol, H2 and H3 are tested together by one comparison, the skill score against FlatANFISMatched with a non-inferiority margin of -0.05 on the lower confidence bound; it is labelled H2/H3 below. The computational part of H2 (training time, memory, latency) was not part of that protocol and is reported separately as an unregistered measurement (Section 4.15).

H4 (threat responsiveness) was not tested as stated: the registered protocols contain no test of it, and the closest procedure, an ablation of x_7 alone run afterwards, is reported as a post hoc, unregistered analysis (Section 4.9).

The remainder of the paper is organised as follows. Section 2 reviews ANFIS, hierarchical fuzzy systems, exploit prediction and container security. Section 3 describes the architecture, the testbed, the realised exploitability-exposure target and the pre-registered forward evaluation. Section 4 reports the observability-overhead measurements, the confirmatory comparison, the ablations and post hoc analyses, the computational-cost measurement, and the forward evaluation. Section 5 discusses the findings and their limits, and Section 6 concludes.

2 Literature Review

2.1 ANFIS

ANFIS is a hybrid algorithm that integrates the learning ability of ANN into the FIS design procedure Jang (1993). ANFIS uses the existing input–output data pairs to tune FIS trainable parameters and generate IF–THEN fuzzy rules. ANFIS architecture consists of antecedent and consequent parts whose parameters are connected by fuzzy

rules based on Sugeno and Tsukamoto-type fuzzy models. The corresponding fuzzy rules with two inputs x_1 and x_2 and one linear output function (f), are given as Jang (1993):

Rule # 1. IF x_1 is A_1 AND x_2 is B_1 THEN

$$f_1 = p_1 \cdot x_1 + q_1 \cdot x_2 + r_1$$

Rule # 2. IF x_1 is A_2 AND x_2 is B_2 THEN

$$f_2 = p_2 \cdot x_1 + q_2 \cdot x_2 + r_2$$

The original learning scheme alternates a least-squares solution of the consequent parameters with gradient steps on the premise parameters Jang (1993). Later work replaced one or both halves of that scheme: subtractive clustering places the initial rules at data-density peaks instead of on a grid Chiu (1994), and population-based or expectation-maximisation learners have been compared with gradient descent on classification benchmarks Jabeen et al. (2023). We use the hybrid scheme with a closed-form ridge step for the consequents and Adam for the premises (Section 3.2), and we use subtractive clustering only to initialise the parameter-matched flat comparator.

The term "hierarchical ANFIS" and the acronym HANFIS are not introduced here for the first time: a hierarchical ANFIS under this name has been applied to speaker verification, with three groups of sound features Soozanchi-K. et al. (2010). The contribution of the present work is not the term. It is the specific domain decomposition (vulnerability, environmental and threat tiers), the temporal, evidence-sealed target definition of Section 3.5, and the accompanying real-world evaluation protocol. Where the distinction matters we refer to the present architecture as HANFIS-DRS (Dynamic Risk Scoring) to avoid conflating the two.

2.2 Hierarchical fuzzy systems and rule-base reduction

A flat fuzzy system with n inputs and m membership functions per input needs up to m^n rules. Raju, Zhou and Kisner showed that arranging the inputs in a hierarchy of low-dimensional subsystems makes the rule count grow linearly in n Raju et al. (1991), and Wang proved that such hierarchies remain universal approximators Wang (1998). Zeng and Keane later gave conditions under which a hierarchical approximation needs fewer rules and parameters than a flat one for the same accuracy Zeng and Keane (2005). Brown and co-authors had already framed the same problem for neurofuzzy models in 1995 Brown et al. (1995). Torra's review catalogues how such hierarchies are built, including the choice of which inputs share a subsystem and how intermediate variables are defined Torra (2002). Lee, Chung and Yu argued that the intermediate variables should keep a readable meaning if the hierarchy is to stay interpretable Lee et al. (2003); our choice of three membership functions on the two intermediate indices follows that argument (Section 3.2), and Section 4.10 reports how far the trained models honour it. Ojha, Abraham and Snášel place hierarchical fuzzy systems among five design families for fuzzy inference and note that the families are often combined Ojha et al. (2019). A more recent survey organises Takagi–Sugeno–Kang fusion into hierarchical, wide and stacked arrangements and treats rule explosion as the main reason for all three Zhang et al. (2024).

Recent applications show the range of the idea and the way it is usually evaluated. Juston and co-authors constrain a Mamdani ANFIS with symmetric, connected trapezoids and a hierarchical rule base, then train it online with deep deterministic policy gradient; the evaluation is a single simulated vehicle run in which tracking error falls over ten episodes Juston et al. (2024). Parthiban and Krishna chain three ANFIS modules, one to classify driving behaviour and two to tune controller gains, and validate each module on a 70/30 split by root mean squared error Parthiban and Krishna (2026). Wu and Zaman build a hierarchical fuzzy controller for robot formation and

compare it in simulation with PID, sliding-mode, fuzzy and ANFIS controllers Wu and Zaman (2025). These studies report point estimates from one run or one split, and none of them compares the hierarchical model with a simple persistence or closed-form baseline on the same data. The present study keeps the hierarchy argument of Raju et al. (1991); Wang (1998) but evaluates it with grouped, embargoed splits, bootstrap intervals, a persistence baseline and a pre-registered forward test.

2.3 Exploit prediction and vulnerability prioritisation

The inputs that carry the skill in this study, exploitability and the KEV indicator, come from a separate literature on which vulnerabilities get exploited. Allodi and Massacci used case-control studies to show that a high CVSS score alone is a weak predictor of exploitation in the wild Allodi and Massacci (2014). Jacobs and co-authors then compared remediation strategies by how many exploited vulnerabilities each one covers for a given patching effort and argued for strategies built on exploit prediction Jacobs et al. (2020). That work led to EPSS, a model that estimates the probability that a vulnerability will be exploited in the next 30 days Jacobs et al. (2021), and to its third version, built with a community special interest group and reported to improve on its predecessors Jacobs et al. (2023). The Stakeholder-Specific Vulnerability Categorization proposed decision trees as an alternative to a single severity number Spring et al. (2021). Koscinski and co-authors compared CVSS, SSVC, EPSS and Microsoft's Exploitability Index on 600 vulnerabilities and found that the systems rank the same vulnerabilities differently, with consequences for teams that rely on one of them Koscinski et al. (2025). Shimizu and Hashimoto combine the three public signals in a fixed order, filtering first by KEV membership or an EPSS threshold and then by CVSS Shimizu and Hashimoto (2026). CISA's Known Exploited Vulnerabilities catalogue is the public record behind the KEV indicator Cybersecurity and Infrastructure Security Agency (2026).

These studies compare scores across vulnerabilities at one point in time. The question asked here is different: given one container image and one week, how much exploitability exposure will it carry over the next four weeks. That framing makes the recent history of the same image a natural predictor, which is why persistence is the main baseline of this study (Section 4.3) and why a formula of the current EPSS and KEV values is a fair competitor for any learned model (Section 4.8).

2.4 Vulnerability risk in container and Kubernetes deployments

NIST's container security guide lists image vulnerabilities, registry hygiene, orchestrator configuration and runtime isolation as the main risk areas of containerised deployments Souppaya et al. (2017). Empirical studies of public images show how large the first of these is. Shu, Gu and Enck scanned Docker Hub images and found vulnerabilities in both official and community images, many of them inherited from the base images they build on Shu et al. (2017). Zerouali and co-authors related outdated packages in Debian-based images to the vulnerabilities and bugs they carry Zerouali et al. (2019). Mills, White and Legg followed 380 images across three scan periods and found that reported vulnerability counts rose despite regular updates, while fewer than one percent of the findings met their criteria for high real-world risk once attack vector and exploit availability were considered Mills et al. (2023). Kim, Park and Hong inspect the file system directly to catch packages that package-manager-based scanners miss Kim et al. (2025). On the orchestrator side, Rahman and co-authors identified eleven categories of security misconfiguration in 2,039 open-source Kubernetes manifests Rahman et al. (2023), and Minna and co-authors examine the security implications of Kubernetes networking Minna et al. (2021). Our earlier Fuzzyfortify framework scored these conditions with expert-weighted fuzzy rules Hafiz Hersyah et al. (2025).

The image-scanning studies above count vulnerabilities; the finding of Mills and co-authors that few of them carry real-world risk is the reason this study weights each CVE by a runtime-reachability estimate ρ and by exploitation evidence rather than by count or severity (Section 3.5). The misconfiguration and networking studies motivate the Kubernetes-exposure input x_5, which this study records as a static rubric value and does not yet derive from telemetry (Section 3.6). What none of these studies provides is a per-image, forward-looking measure of exploitability exposure built from sealed public evidence, which is the target defined in Section 3.5.

3 Methodology

3.1 HANFIS architecture and risk formulation

This study adopts a controlled, repeated-measures experimental design to develop and validate an evidence-based Hierarchical Adaptive Neuro-Fuzzy Inference System (HANFIS) for dynamic risk scoring in a cloud-native environment. The methodological premise is that runtime telemetry cannot be incorporated into a cybersecurity risk model without first establishing its provenance, measurement cost, stability, and reproducibility. Otherwise, resource consumption or latency introduced by the observability stack itself could be misinterpreted as a change in workload exposure and, consequently, as a change in cyber risk.

The study therefore separates two related but analytically distinct objectives. The first objective is to quantify the overhead of Kubernetes observability. The second is to establish which validated runtime measurements are stable enough to be considered as inputs for the environmental subsystem in future work; in the present study the environmental inputs are static per-asset values (Section 3.5). The overhead experiment compares the same baseline workload under two conditions: monitoring enabled (ON) and monitoring disabled (OFF). The workload, service configuration, request path, cluster topology, sampling procedure, and validation gates are held constant.

Monitoring state is the principal experimental factor. This design provides a control condition against which the incremental cost of telemetry collection can be estimated.

The observability experiment comprises ten paired runs (Runs 7–16). Conditions are counterbalanced across runs using ON–OFF and OFF–ON sequences. Counterbalancing reduces systematic error from execution order, cache warming, temporal load variation, and cluster stabilisation. Each transition is guarded by a preflight or post-transition validation step. A condition is admitted to measurement only when the Kubernetes nodes, baseline workload, HTTP service, and required monitoring state satisfy predetermined acceptance criteria. Failed gates produce a `HOLD` decision rather than an experimental observation. This prevents measurements obtained from a degraded or incorrectly configured system from entering the analytical dataset.

3.2 HANFIS architecture, rule count, and parameter complexity

HANFIS composes three first-order Sugeno ANFIS panels. The vulnerability panel $V(x_1, x_2, x_3) \to v$ takes CVSS, Exploitability, and Patch Risk. The environmental panel $E(x_4, x_5, x_6) \to e$ takes MFA Risk, Kubernetes Exposure, and Asset Criticality. The integration panel $I(v, e, x_7) \to \tilde{r}$ takes the two intermediate indices and Threat Intelligence, and the final score is $r = \mathrm{clip}(\tilde{r}, 0, 1)$. No panel is given an intermediate target: v and e are latent, and gradients flow end-to-end from the loss on r back through both tiers. Figure 1 shows the structure.

Each panel uses Gaussian membership functions,

$$\mu_{ij}(z_i) = \exp\left(-\frac{(z_i - c_{ij})^2}{2\sigma_{ij}^2}\right), \qquad i = 1, \ldots, n,\ j = 1, \ldots, m, \tag{3}$$

Sugeno-normalised firing strengths $\bar{w}_k = w_k / \sum_l w_l$ with $w_k = \prod_i \mu_{i,j_k(i)}(z_i)$, and first-order consequents $f_k(z) = \beta_{k0} + \sum_i \beta_{ki} z_i$, so that a panel's output is $\sum_k \bar{w}_k f_k(z)$.

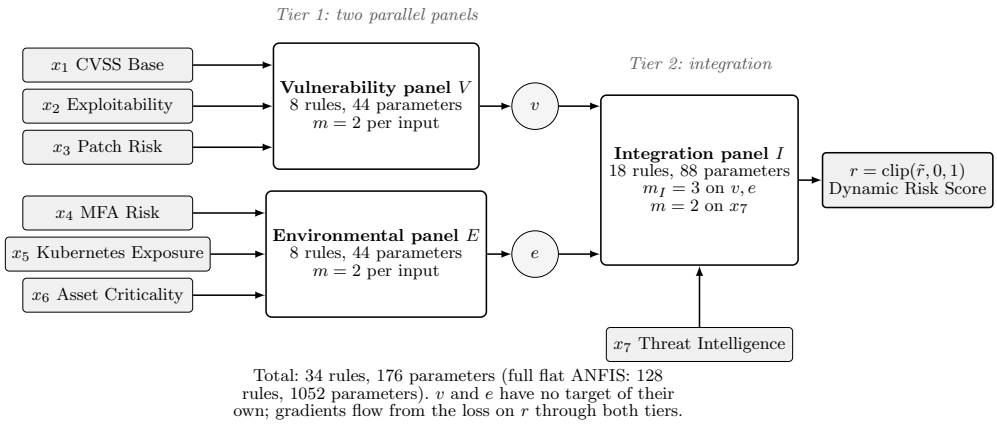


Fig. 1 Structure of HANFIS. Two first-tier Sugeno panels, vulnerability V and environmental E, produce the latent indices v and e. An integration panel I combines them with Threat Intelligence x_7 into the Dynamic Risk Score r. Rule and parameter counts per panel are those of Table 1.

The raw inputs $x_1, \ldots, x_7$ each use $m = 2$ membership functions; the intermediate variables v, e inside the integration panel use $m_I = 3$ (amendment A-02), which turns v and e into the linguistic categories {low, medium, high} rather than a single coarse split, following the requirement that reduced intermediate variables remain semantically interpretable (Lee, Chung, and Yu Lee et al. (2003)); Section 4.10 shows that the trained centres of v and e do not always keep that order. A connected trapezoidal (Ruspini) membership family is available as an ablation alternative to the free Gaussian family (Section 4.5).

Table 1 reports the resulting rule and parameter counts against a full flat ANFIS over the same seven inputs with $m = 2$ membership functions each. The hierarchy reduces the rule count by 73.4% and the parameter count by 83.3%; unlike the flat model, rule growth is linear in the number of panels rather than exponential in the number of inputs Raju et al. (1991); Wang (1998).

Consequent slopes in every panel are constrained non-negative through a softplus reparameterisation (amendment A-04), so that each rule's consequent is non-decreasing in every input under the stated higher-is-riskier convention. Intercepts stay free and rule weights are normalised, so r itself is not guaranteed to be monotone.

Table 1 Rule and parameter counts by panel ($m = 2$ on raw inputs, $m_I = 3$ on v, e) against a full flat ANFIS over the same seven inputs.

Panel	Inputs	Rules	Premise params.	Consequent params.	Total
V	x_1, x_2, x_3	8	12	32	44
E	x_4, x_5, x_6	8	12	32	44
I	v, e, x_7	18	16	72	88
HANFIS (total)	7 raw + 2 intermediate	**34**	40	136	**176**
Flat ANFIS (reference)	7 raw	128	28	1024	1052

Monotonicity is therefore checked empirically (Section 4.7). Training is hybrid (amendment A-06): the integration panel's consequents are solved in closed form by ridge least squares every epoch,

$$\beta = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top y, \qquad \lambda \geq 10^{-3}, \tag{4}$$

with Φ the per-rule design matrix $\bar{w}_k \cdot (1, z_1, \ldots, z_n)$, while premise parameters are updated by Adam. An alternating variant (E2) additionally solves both lower panels' consequents in closed form every epoch, by freezing the integration panel's firing weights and consequents so that the final output becomes affine in v and e per row, which in turn makes it linear in the lower panels' own consequents given their (frozen-premise) firing weights; this reduces to Adam training premise parameters only. Three pre-registered restarts with fixed seeds are trained per fold, patience 100 on validation loss with a minimum of 200 epochs, and the restart with the lowest validation loss is kept (amendment A-09); a pure end-to-end Adam variant without any closed-form step is retained as an ablation (Section 4.5) because it is unstable on some seeds without the closed-form anchor.

3.3 Experimental platform and telemetry validation

The experimental workflow was implemented in Ubuntu 24.04 under Windows Subsystem for Linux 2 (WSL 2), with Docker Desktop providing the container runtime and Kubernetes integration. Shell procedures were stored as versionable `.sh` files so that every experimental gate could be re-examined without reconstructing the command sequence. The Kubernetes testbed comprised two Ready nodes and a baseline application, P0, deployed with two ready replicas in the `hanfis-research` namespace; a load-generator pod accessed P0 through an internal service, and status code 200 was required at each critical gate. The observability stack (Prometheus The Prometheus Authors (2026), Prometheus Operator, kube-state-metrics, and node-exporter) ran in a separate `monitoring` namespace; in the validated ON state it held five fully ready pods, and in the OFF condition the workloads were scaled to zero and the exporter DaemonSet made unschedulable, with the original definitions saved so that the ON state could be restored deterministically.

System preparation was organised as a sequence of auditable gates covering the baseline workload, cluster health, the observability deployment, the measurement procedure, protocol freezing and SHA-256 manifests (Table S1 of Additional file 1). The first gates of the protocol-development phase (O5–B4.10 to O5–B4.11B) describe the procedure only, and since one of their gate reports could not be reconstructed they are not cited as evidence of passing. The evidence that the gates were satisfied is the set of run archives and manifests for Runs 7–16 (Section 3.4). Because P0 stayed at two of two ready replicas with HTTP status 200 across every admitted transition, the change in observability state did not itself interrupt the target service; the full description is in Supplementary note S1 of Additional file 1.

3.4 Observability-Overhead Protocol (external-v3.0)

The gates and integrity controls of Table S1 of Additional file 1 were developed during the first runs and were carried forward unchanged into protocol `external-v3.0`, which produced all observability-overhead results reported in Section 4.1. The protocol comprises ten complete counterbalanced ON–OFF pairs (Runs 7–16): Runs 8, 10, 12, 14 and 16 were executed OFF–ON, and Runs 7, 9, 11, 13 and 15 ON–OFF. The earlier Runs 1–6 belong to the protocol-development phase and are not pooled with the ten pairs. One aborted attempt of Run 7 and one aborted attempt of Run 17 failed the integrity gate and are recorded as incidents rather than observations.

The protocol follows a fail-closed rule. A condition is admitted to measurement only when the nodes, the P0 workload (two of two replicas), the HTTP service (status 200) and the required monitoring state satisfy the acceptance criteria of Table S1 of Additional file 1. A failed gate yields a `HOLD` decision, not an observation. Each run is archived with a SHA-256 manifest, so that the ten run archives and the cumulative analysis can be re-verified without reconstructing the original command sequence.

Telemetry preserves the link between each value and its Kubernetes object, node, container, condition and run. The measurements are node-, pod- and container-level CPU and memory, pod readiness, HTTP outcome and response latency; identical request-generation and sampling procedures are applied in the ON and OFF conditions. Configuration files, event logs, gate reports and manifests document how each observation was produced and are not treated as HANFIS training features.

Let $Y_{r,c,k}$ denote observation k of run r under condition $c \in \{\mathrm{ON}, \mathrm{OFF}\}$, where Y may be CPU, memory or latency. The run mean is

$$\bar{Y}_{r,c} = \frac{1}{n_{r,c}} \sum_{k=1}^{n_{r,c}} Y_{r,c,k}, \tag{5}$$

the paired absolute overhead is $\Delta Y_r = \bar{Y}_{r,\mathrm{ON}} - \bar{Y}_{r,\mathrm{OFF}}$, and the relative overhead, when the OFF denominator is non-zero, is

$$\Delta Y_r^{(\%)} = \frac{\bar{Y}_{r,\mathrm{ON}} - \bar{Y}_{r,\mathrm{OFF}}}{\bar{Y}_{r,\mathrm{OFF}}} \times 100\%. \tag{6}$$

The run, not the individual sample, is the independent replication unit, which avoids pseudoreplication from temporally adjacent samples. Inference over the ten paired differences uses an exact two-sided sign-flip test, whose smallest attainable p with ten pairs is $2/2^{10} = 0.002$, together with a 10,000-resample bootstrap Efron (1979) confidence interval (fixed seed) for the mean difference. Effect sizes, confidence intervals and the consistency of direction carry more interpretive weight than a binary significance decision. The estimated increment is an instrumentation cost; it must not be read as a change in application exposure.

3.5 Realised exploitability-exposure target and real-world asset-week dataset

The environmental inputs x_4 to x_6 are static per-asset values recorded in the asset inventory. Input x_5 follows a written rubric over the cluster configuration (whether the asset accepts traffic from many pods, is scoped by a NetworkPolicy, or holds admission rights) and x_6 a written three-factor rubric. The runtime telemetry described above is not a model input, so the time variation of the score comes from the EPSS and KEV signals. The label used to train and evaluate HANFIS is constructed separately, from realised vulnerability outcomes rather than expert judgement. The observation unit is an asset-week: one digest-pinned container image a observed at one Monday t. For a 30-day horizon $\Delta = 30$ discretised into four weekly checkpoints

$T = \{t{+}7, t{+}14, t{+}21, t{+}28\}$,

$$y(a,t) = \frac{100}{|T|} \sum_{\tau \in T} \max_{v} \left[X_v(\tau)\, S_{v,a}(\tau) \right], \tag{7}$$

$$\begin{aligned} X_v(\tau) &= \max\left(\mathrm{EPSS}_v(\tau), \mathbb{1}[v \in \mathrm{KEV}(\tau)]\right), \\ S_{v,a}(\tau) &= \rho_{v,a} \cdot \mathbb{1}[v \text{ affected and reachable on } a \text{ at } \tau], \end{aligned} \tag{8}$$

where $\rho_{v,a} \in \{0, 0.1, 0.25, 0.6, 1\}$ is a five-level runtime-reachability estimate (not installed; installed with no dependency; transitively loaded but inactive; loaded with an unproven trigger; loaded with a demonstrated trigger). S uses ρ rather than a bare indicator because otherwise the label saturates at 100 once any affected CVE enters KEV and the positive control described below would not differ between its two conditions. A CVE is treated as active for image a at τ only if its publication date is $\leq \tau$ and its version-range status is `AFFECTED`, so no feature or label uses information not yet public at the time it purports to describe; y is left unlabelled (feature-only) when T extends beyond the last sealed evidence snapshot.

Evidence is sealed weekly: one EPSS bulk file per Monday and one CISA KEV catalogue Cybersecurity and Infrastructure Security Agency (2026) download, each stored byte-for-byte and hashed (SHA-256) into a manifest that a verification step rechecks before any build. This sealed 57 weekly snapshots (2025-08-25 to 2026-09-21) and a KEV catalogue of 1,728 entries. Each image contributes one frozen, digest-pinned Trivy Aqua Security (2026) vulnerability scan. CVEs lacking a publication date (a known Debian/Ubuntu-tracker gap) are dated by the first sealed weekly snapshot in which they are scored, and a CVE that never appears in any sealed snapshot is excluded from the active set rather than assumed absent. A positive control, P0-IF, isolates one asset with the image-filter Nginx module either inactive ($\rho = 0.25$: transitively present, unloaded) or active with its trigger path exercised ($\rho = 1$). It is built

from the same digest-pinned base image as the primary asset P0, so its vulnerability surface (109 of 135 known CVEs as of the freeze date, 25 in `libexpat`) is not an artefact of an unrelated rebuild.

At the time of writing the inventory holds 120 assets (117 images across web, proxy, cache, database, queue, runtime, base, observability, and Kubernetes-native infrastructure and security tooling categories, plus P0 and the two P0-IF conditions) observed across the 57-week window. It comprises 6,833 asset-week rows, after excluding seven rows whose evidence remained unresolved (one CockroachDB CVE under Trivy status `under_investigation` at the inventory freeze date); the non-missing rate is 0.93, and no row is left in an unresolved evidence state. This clears the pre-registered data-quality gate: protocol frozen and dated; ≥ 10 observations per model parameter, i.e. $\geq$ 1,320 rows for the 176-parameter configuration; no leakage across the split described below; and every mandatory baseline runnable on the pilot. We treat the gate as a necessary condition for training HANFIS on real data, not as a result in itself. The three open decisions on label aggregation, label scale, and the definition of Patch Risk were closed by cross-checking the decomposition above against this subsection's own formula; they were not treated as free parameters. The row-wise maximum and the raw 0–100 scale are the definition as written, not a choice, and the {0,1,2} Patch Risk contract already matches the running pipeline. The 120-asset inventory described here is the final inventory used for every result in Section 4; no further scaling of the inventory is planned for this paper.

3.6 Relation of validated telemetry to HANFIS

The proposed risk model contains seven conceptual inputs arranged hierarchically (Section 3.2). Runtime telemetry is not an input of the model evaluated here: x_5 is a static rubric value (Section 3.5), and raw CPU, memory and latency measure resource use, not exposure. Deriving x_5 from runtime telemetry would require a mapping from

raw measurements to an exposure score that is specified and pre-registered before any model evaluation, to avoid post hoc feature selection; we leave it as a separate hypothesis for future work.

Figure S1 of Additional file 1 summarises the relationship among infrastructure preparation, controlled telemetry acquisition, validation, and HANFIS. The figure is supplied as native LaTeX/TikZ so that it remains editable in the Springer manuscript.

3.7 Validity safeguards and methodological boundary

Internal validity rests on paired comparisons, a stable workload, condition gates, health checks, counterbalanced order, and the rejection of measurements taken under invalid states. Construct validity is protected by separating observability overhead from cybersecurity exposure and by requiring a semantic mapping before any telemetry variable is used as a HANFIS feature. Reliability is supported by script-based execution, frozen configuration and cryptographic integrity checks, and traceability runs from raw samples through summaries to model-ready features.

Several limits remain. WSL 2 with Docker Desktop is reproducible but does not represent every production Kubernetes deployment, and the two-node topology and P0 workload constrain external validity. Across the ten pairs of Runs 7–16 (external-v3.0), the two nodes together used on average about 230 mCPU without and 329 mCPU with the monitoring stack (node-level CPU summed over both nodes). That is 1.4% and 2.1% of the host's 16 logical cores; the capacity was taken from the v4.2 collector metadata and `docker info` because it was not logged during these runs. The absence of a latency effect was therefore observed at these utilisation levels and need not extend to saturated nodes. Ten pairs make an effect-size reading appropriate, but they bound the smallest attainable sign-flip p at 0.002 and give limited power for small latency effects. The protocol was frozen before the ten pairs were analysed, yet its

version is entangled with the platform on which the earlier runs were taken, so cross-platform comparisons are descriptive. Finally, successful telemetry collection does not by itself show predictive value for cyber risk; that claim rests on the held-out HANFIS evaluation against the baselines of Section 4.

3.8 Prospective evaluation (E7): pre-registered design

The confirmatory comparison and the post hoc analyses reported above use data whose labels were already known when the questions were asked. To obtain an evaluation that does not, we froze a forward-evaluation protocol before any label for the test weeks was computed. The protocol, the five final HANFIS models (trained only on labelled weeks up to 2026-07-27, the first test week minus the 30-day embargo), the blend weight, the coefficients of the max-form formula of Section 4.8, and all analysis scripts were committed and tagged (`e7-prereg-2026-10-12`, created on 2026-10-12 at 15:04 +0700). A script verified that the tagged tree is identical to the working files. The evaluation script refuses to run unless this verification passes, and it runs only once. The protocol was also registered on the Open Science Framework before any forward label was computed (registration date 2026-10-13; `https://osf.io/ahcvs/`; DOI `10.17605/OSF.IO/AHCVS`). The analyses E14c and E15 to E20 were run on earlier data after the freeze and before the registration; the registration lists E15 to E20 and does not list E14c.

The test weeks are the Mondays t from 2026-08-31 with $t + 28$ days not later than the data date D_2 = 2026-11-02, six weeks in total; fewer than six matured weeks postpone the study rather than relax its criteria. Weekly EPSS and CISA KEV snapshots after 2026-09-21 are downloaded only after the tag, the vulnerability database is frozen once on D_2, and every image is rescanned with it. The predictors are HANFIS (mean of five seeds), persistence (the label of week $t - 4$), a fixed blend of the two ($\lambda = 0.5109$, fitted on out-of-fold predictions), and the formula. Skill scores against

persistence use pooled mean squared error with a bootstrap clustered by image (2,000 resamples). Two families of one-sided hypotheses are tested, each with its own Holm correction Holm (1979) at $\alpha = 0.025$: whether HANFIS and the blend improve on persistence (P7a, P7b), and whether the formula improves on persistence while HANFIS is not better than the formula (P7e, P7f). The direction of predicted change is a secondary hypothesis (P7d). Rows with unresolved vulnerability-range evidence are dropped only if they amount to at most 2% of the test-week rows, and an asset that loses all of its test weeks leaves the analysis and is reported (`cockroachdb` was expected to, but no asset was dropped). The results are reported in Section 4.16.

Use of AI tools

An AI assistant (Claude, Anthropic; the Claude Sonnet 5.5, Claude Opus 5.5 and Claude Fable 5.1 models, used through the Claude Code environment during 2026) helped to write and test analysis and utility code, to draft the two unregistered protocols (Sections 4.15 and 4.9), to run scripts at the authors' direction, including the stages of the forward evaluation on the pre-registered date, to draft and revise manuscript text, including first drafts of parts of the Results and Discussion, and to check the reference list against doi.org. The authors decided every protocol, hypothesis and claim before the corresponding run, verified the code, the results and the references, reviewed and edited all text, and take full responsibility for the content. The assistant is not an author (see Declarations).

4 Results

4.1 Observability overhead

Ten complete run pairs passed every acceptance gate (Runs 7–16, protocol `external-v3.0`, counterbalanced five ON–OFF and five OFF–ON). One aborted attempt of Run 7 and one aborted attempt of Run 17 failed the integrity gate and

were recorded as incidents rather than observations, in accordance with the fail-closed rule stated in Section 3. Table 2 reports the paired differences $\Delta Y_r = \bar{Y}_{r,\mathrm{ON}} - \bar{Y}_{r,\mathrm{OFF}}$.

Table 2 Paired monitoring overhead over ten counterbalanced runs (protocol external-v3.0). CI: 10,000-resample bootstrap; p: exact two-sided sign-flip test (floor $2/2^{10} = 0.002$).

Measure	Mean Δ	95% CI	Relative	p
Resource cost of the observability stack				
Node CPU (mCPU)	+99.36	[96.29, 103.35]	+43.3%	0.002
Monitoring namespace CPU (mCPU)	+39.79	[33.00, 52.77]	—	0.002
Node memory (MiB)	+280.76	[250.99, 303.54]	+18.7%	0.002
Monitoring namespace memory (MiB)	+204.19	[191.38, 218.82]	—	0.002
Effect on the observed workload				
Latency median (ms)	−0.005	[−0.055, +0.044]	−0.06%	0.859
Latency P95 (ms)	+0.059	[−0.126, +0.265]	+0.67%	0.607
Latency P99 (ms)	+0.188	[−0.127, +0.448]	+1.94%	0.268
P0 container CPU (mCPU)	−0.059	[−0.156, +0.041]	−3.9%	0.305
P0 container memory (MiB)	−0.334	[−0.522, −0.147]	−1.19%	0.016

Two results follow. First, the resource cost of the observability stack is large, consistent, and unambiguous: every pair shows the same direction and the effect sizes are extreme ($d_z = 16.5$ for node CPU). Node-level CPU increases by 99.4 mCPU while the monitoring namespace itself accounts for only 39.8 mCPU, so pod-level accounting understates the true cost by a factor of approximately 2.5; the remainder is induced platform work (kubelet, cAdvisor, container runtime, and the API server serving the additional scrape traffic). The same pattern holds for memory (280.8 MiB at node level against 204.2 MiB inside the namespace).

Second, this resource cost does not propagate to the observed workload. For median latency and P95 latency we fail to reject the null hypothesis, with confidence intervals of $[-0.06, +0.04]$ ms and $[-0.13, +0.27]$ ms respectively, that is, below 3% of the baseline latency. The P0 container consumes marginally *less* memory under monitoring (-0.33 MiB, $p = 0.016$), an effect of practical size zero that we report for completeness rather than interpret. With ten pairs the sign-flip test can reach $p = 0.002$, so the

failure to reject on latency reflects the absence of a detectable effect rather than the absence of statistical power.

These measurements justify treating the observability stack as a fixed resource overhead that must be budgeted for, and they show no latency change that would need correcting if telemetry were later used as a model input. We do not claim equivalence: the confidence intervals bound the effect; they do not exclude it.

4.2 Training configuration

Throughout, "pre-registered" means that a dated protocol was committed to the project's version-controlled repository and tagged before the corresponding training run. This does not refer to an entry in an external registry, except for the forward evaluation (Section 3.8), which was also registered on the Open Science Framework. The confirmatory protocol was committed on 30 September 2026 and tagged the same day (tag `e4-prereg-2026-09-29`, whose name carries the nominal date of the protocol file). The tag covers the protocol, the seed list, the split code and the HANFIS model code. The parameter-matched flat comparator and the training script were committed on 30 September and 1 October 2026, after the tag but before the first confirmatory run on 1 October 2026 (109-asset inventory); the 120-asset run reported here took place on 2 October 2026. Later additions to the training script (saving models, recording the normalisation) do not change its predictions.

All results below use the frozen configuration of amendment A-09: hybrid closed-form/Adam training (Section 3.2), learning rate 10^{-2}, ridge 10^{-3}, three pre-registered restarts per fold with fixed seeds, patience 100 on validation loss with a minimum of 200 epochs, the Gaussian membership family with the monotonicity constraint active (justified in Section 4.5). Evaluation uses a grouped, embargoed temporal split (`GroupTimeSplit`, amendment A-16). Images are assigned to n held-out groups by a

seeded permutation. Fold k trains on every other group up to a cutoff date and evaluates group k inside the following block of weeks, after a 30-day embargo equal to the outcome window. No image and no overlapping outcome window therefore appears on both sides of any fold, and every image is evaluated exactly once. On the completed 120-asset inventory this yields 5 folds of 95–96 training images against 23–24 held-out images. The alternating lower-panel closed-form training variant (Section 3.2) was verified against an independent closed-form derivation on synthetic data (mean squared error $0.78 \to 4 \times 10^{-4}$ over six alternating iterations with no premise training at all) before being adopted as amendment A-09's frozen training scheme, used throughout the real-data results reported below and in Section 4.4.

4.3 Predictive performance

Table 3 reports the mandatory baselines on the 6,357 labelled rows described in Section 3.5, pooled over all five test folds. Pooling gives one mean-squared-error ratio over every held-out row, instead of an average of per-fold ratios, which is unstable when a fold's reference error is near zero. $\mathrm{SS}_{\mathrm{clim}}$ and $\mathrm{SS}_{\mathrm{pers}}$ are skill scores against the training-fold climatology and against a causal 4-week persistence forecast, respectively.

Table 3 Mandatory baselines, pooled over 5 grouped-temporal folds, raw label scale (0–100).

Baseline	RMSE	$\mathrm{SS}_{\mathrm{clim}}$	$\mathrm{SS}_{\mathrm{pers}}$
Climatology	4.010	0.000	−37.78
Single CVSS score	3.537	0.222	−29.17
Persistence (causal, 4-week lag)	0.644	0.974	0.000
Ridge regression, all 7 inputs	1.169	0.915	−2.293

A single CVSS score explains a modest fraction of the realised label's variance ($\mathrm{SS}_{\mathrm{clim}} = 0.222$). That is markedly more than on the earlier 27-image pilot

($SS_{clim} \approx 0$), though still far below persistence or ridge. Static severity alone therefore remains a poor descriptor of which vulnerabilities become exploitable in practice, even as the larger inventory gives it some traction. A naive persistence forecast, in contrast, recovers 97% of the climatology-relative skill: the label is strongly autocorrelated because the set of active CVEs and their EPSS scores change slowly week to week. On the completed 120-asset inventory, a linear ridge model over all seven inputs no longer improves on persistence and is now substantially worse ($SS_{pers} = -2.293$ pooled, raw scale), a reversal from the 27-image pilot's +0.301, confirmed on every one of the five grouped-temporal folds rather than driven by one outlier (Section 5.1). This is consistent with a linear model over-extrapolating as the inventory's heterogeneity grew, where persistence's week-to-week copy remains comparatively safe. Persistence is therefore the bar hypotheses H1b and H1c (Section 1) ask HANFIS to clear or match; ridge alone no longer clears it either. The HANFIS-versus-persistence, HANFIS-versus-ridge, and HANFIS-versus-FlatANFISMatched comparisons (H1a–H1c and H2/H3, which the frozen protocol tests together) are pre-registered under a frozen protocol (fixed seeds, fold assignment, smallest-effect-size-of-interest of 5% on SS_{pers}, group-clustered bootstrap confidence intervals, Holm correction across the family of four comparisons). Table 4 reports the outcome on the 120-asset inventory described in Section 3.5: HANFIS confirms H1a, H1c, and H2/H3, does not confirm H1b, and H2′ is reported as the capacity reference, without a decision criterion. Fuzzyfortify is not part of the statistical family: it requires four native inputs (complexity-resilience index, impact, base score, exploitability) that the asset-week data do not produce, and the comparison protocol forbids unreported proxy mappings. We therefore neither use it as a baseline nor report a Fuzzyfortify comparison.

The H1a comparator, a single CVSS score, has no access to exploitability or the KEV indicator, the two inputs that carry HANFIS's skill (Section 4.8), and the H1c comparator, a ridge model on all seven inputs, is itself worse than persistence (Table 3).

Table 4 E4 confirmatory results: HANFIS skill score against each comparator, group-clustered bootstrap 95% CI, Holm-corrected across the four-hypothesis family.

Hypothesis	Comparator	SS	95% CI	Outcome
H1a	Single CVSS score	0.967	[0.951, 0.980]	Confirmed
H1b	Persistence (4-week)	0.015	[−0.607, 0.246]	Not confirmed
H1c	Ridge, 7 inputs	0.696	[0.527, 0.816]	Confirmed
H2/H3	FlatANFISMatched	0.665	[0.269, 0.844]	Confirmed
H2′	Flat ANFIS (full)	0.253	—	Reported, no criterion

H1b's confidence interval crosses zero, so HANFIS is not distinguishable from a causal 4-week persistence forecast. The E3 pilot showed the same pattern (Table 3): the strongly autocorrelated label remains the hardest bar in this study. The post hoc analysis in Section 4.5 indicates why: HANFIS outperforms persistence when the label moves and underperforms it when the label is stable.

The H2/H3 margin is large for two models holding an identical 176-parameter budget, but it is not uniform across folds. HANFIS wins on four of the five held-out blocks, with per-fold skill against FlatANFISMatched ranging from 0.165 to 0.924. The fifth fold (fold 1) is negative (SS = −0.122). It coincides with a secondary diagnostic in which HANFIS's predicted variance exceeds the label's own variance by more than three orders of magnitude on several near-constant-label images in that fold (ratio $> 1{,}000\times$ in the most extreme case). This is consistent with the model under-damping day-to-day EPSS/CVSS snapshot noise where the weekly aggregate carries little real signal, a limitation discussed further in Section 5.1; it is not a single outlier fold driving an otherwise uniform result.

FlatANFISMatched's held-out predictions nonetheless range well outside the plausible label interval (down to −6.679 and up to 11.059, against HANFIS's 0.000–10.401). This is consistent with poor extrapolation from unsupervised cluster centres that have no access to the label's own generative grouping, which is the vulnerability/environmental/threat decomposition that HANFIS's architecture encodes by construction (Section 3.2). We read this as suggestive of the mechanism claimed in C1 and H2/H3,

not as a methodological artefact: the random 15% validation carve-out used only to select the best of three restarts is identical for both models, while the reported skill score itself comes from the temporally and group-disjoint held-out block that the carve-out never touches.

4.4 Accuracy-complexity curve

The scalar inter-tier bottleneck between the lower panels and the integration panel is a candidate explanation for any residual gap between HANFIS and a full-capacity flat ANFIS: compressing v and e to a single number may discard cross-subsystem interaction that the flat model can still represent. A sweep over the bottleneck width k, widening v and e from scalars to k-dimensional vectors passed into the integration panel, traces this trade-off directly. Grid-partitioning the integration tier, however, makes its rule count grow as $m_I^{2k} \cdot m$. At the frozen $m_I = 3$, $k = 1$ reproduces HANFIS exactly (34 rules, 176 parameters), $k = 2$ already costs 178 rules and 1,152 parameters (6.5x), and $k \geq 3$ exceeds 1,400 rules (Figure 2), which is more parameters than the present inventory has rows. The sweep script refuses these settings by default rather than attempting an underdetermined fit. We ran $k \in \{1, 2\}$; $k = 4$, originally planned, was dropped on these grounds. This combinatorial explosion is itself read as evidence for why a scalar bottleneck was the design choice in the first place, independent of what $k = 1$ vs. $k = 2$ shows empirically.

At k=1 the sweep uses exactly A-09's training scheme: the alternating closed-form least squares for both lower panels, the intercept refit after the monotone projection, and the same hard clamp on v, e that HANFIS's own forward pass applies. It is implemented separately in `hanfis_bottleneck.py` under a multi-head representation that generalises cleanly to $k > 1$. A per-row comparison with the confirmatory predictions on the earlier 109-asset inventory confirmed this reproduction directly (Pearson $r = 0.9984$ on point predictions, identical fold assignment for all 981 held-out rows);

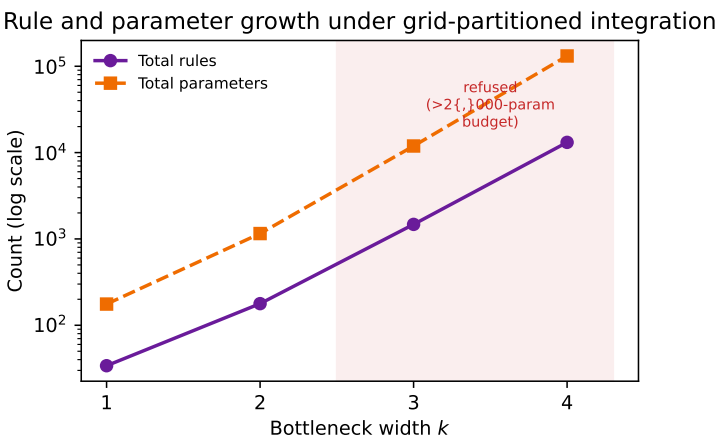


Fig. 2 Rule and parameter growth with bottleneck width k under grid partition of the integration tier. The shaded region ($k \geq 3$) exceeds the parameter budget the present inventory supports and was refused rather than attempted.

the comparison was not repeated on the 120-asset inventory. An earlier version of this sweep used a simpler training scheme (the Integration-only closed form rather than the alternating scheme) and omitted the intercept refit after the monotone projection; both have since been corrected, and the numbers below supersede that earlier, confounded comparison.

Table 5 E5 bottleneck-width sweep: pooled RMSE and skill against persistence, 95% confidence intervals from 2,000 image-clustered bootstrap resamples (the same procedure as Table 4), 120-asset inventory.

k	Rules / Params	RMSE (pooled)	$\mathrm{SS}_{\mathrm{pers}}$ [95% CI]
1	34 / 176	0.587	0.169 [−0.330, 0.343]
2	178 / 1152	0.820	−0.622 [−2.259, −0.100]

k=1 recovers skill broadly consistent with HANFIS's own H1b result (Table 4: $\mathrm{SS}_{\mathrm{pers}} = 0.015$, $[-0.607, 0.246]$). Both confidence intervals cross zero and overlap substantially, as the per-row parity above predicts, though the point estimates (0.169 vs. 0.015) agree less closely than on the smaller inventory. k=2's interval now excludes

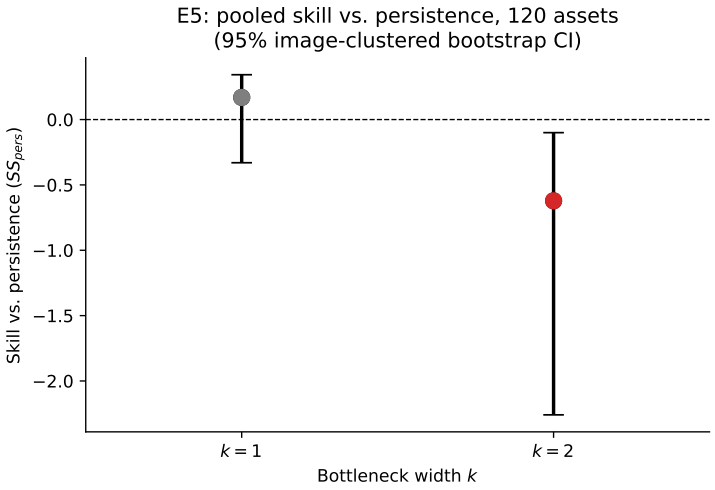


Fig. 3 E5: pooled skill against persistence for $k \in \{1, 2\}$ with 95% image-clustered bootstrap confidence intervals. k=1's interval now crosses zero; k=2's is entirely negative.

zero entirely on the negative side. On the completed 120-asset inventory, widening the bottleneck is significantly worse than persistence. On the smaller inventory it had been unsupported by the evidence but not shown to be worse. The paired comparison on matched resampled images (the same resample is used for both arms, so sampling noise is shared) gives $\mathrm{SS}_{\mathrm{pers}}(k{=}1) - \mathrm{SS}_{\mathrm{pers}}(k{=}2) = 0.791$, 95% CI $[0.354, 2.037]$, which excludes zero (100% of resamples favour k=1). k=1 is also pooled-RMSE-better on all five folds (two-sided sign test $p = 0.0625$, the floor attainable with five folds). This is the same margin of evidence as on the smaller inventory, even though each arm's marginal significance against persistence flipped in opposite directions.

This is the strongest statement the present inventory supports. It is a mechanism claim, not a claim that wider bottlenecks are intrinsically worse: every k=2 fold falls under ten observations per parameter (range 0.3–2.9), the same ten-per-parameter threshold applied elsewhere in this study (Section 3.5), while k=1 clears it in three of five folds. A secondary diagnostic separates part of this gap from pure data scarcity. Disabling the hard clamp on v, e (an ablation, not the model HANFIS reports) costs k=1 0.161 of skill but costs k=2 0.813. Both were re-run on the completed 120-asset

inventory; the costs were 0.019 and 0.394 respectively on the 27-image pilot. This is consistent with v, e being estimated less reliably from the same training rows once six times as many consequent parameters compete for them, so that excursions outside $[0, 1]$ are both more frequent and more consequential once clamped. Separating an intrinsic cost of wider bottlenecks from this data-scale effect requires an inventory substantially larger than the 120 assets collected here, which is left for future work. The architecture's own combinatorial growth in k (Figure 2) means that this separation gets harder, not easier, as k increases.

4.5 Ablation and robustness analysis

Table S2 of Additional file 1 reports the membership-family×monotonicity ablation (E1) on two synthetic scenarios (S1 smooth, S2 with a hard threshold; five seeds per cell). The monotonicity constraint costs little on the smooth scenario but is expensive on the threshold scenario for the Gaussian family (RMSE 0.117 → 6.067), because softplus-constrained slopes cannot represent a local reversal; the connected Ruspini family degrades far less (0.646 → 2.126). Amendment A-10 therefore keeps the Gaussian family with monotonicity active as the frozen default and treats Ruspini as a documented, cheaper fallback. An end-to-end pure-Adam variant without the closed-form anchor (Section 3.2) diverged on a subset of seeds, confirming that the hybrid scheme is necessary.

Sensitivity to the EPSS model change. The EPSS scoring model changed version on 2026-06-15 (`v2025.03.14` to `v2026.06.15`). In a post hoc sensitivity analysis, the confirmatory pipeline was re-run on the 4,560 asset-weeks whose labels all pre-date the change (720 held-out rows; Table S3 of Additional file 1). H1a, H1c and H2/H3 remain confirmed, and H1b remains unconfirmed, with a much wider interval.

Where HANFIS and persistence differ. The pooled skill against persistence (0.015) hides two regimes (exploratory, post hoc, outside the Holm-corrected family). For the

four weeks in which the label is fully computed under the new EPSS model while the lagged label is still computed under the old one (2026-W24 to W27; 95 rows), skill against persistence is +0.483; on the remaining 985 rows it is −0.420. HANFIS is therefore reactive rather than predictive: better than persistence when the label moves, worse when it does not. Two further observations qualify H2/H3. The first held-out fold, trained on 2.2 observations per parameter, contributes 91% of the pooled gap to the matched model; without it the skill against FlatANFISMatched is 0.183 and against the full flat model 0.084. The fold-by-fold and week-by-week values are in Supplementary note S2 of Additional file 1.

4.6 Explanation fidelity (E6)

We tested whether the rule-level explanation that HANFIS exposes is faithful to its inference, using the 25 saved models (five folds × five seeds), which reproduce the confirmatory predictions exactly. The version-control freeze preceded the commit that contained the protocol text, so the protocol is not frozen in the strict sense and the run is reported as a documented deviation; Table S4 of Additional file 1 lists the outcome. The rule-attribution check failed its criterion: removing the three rules ranked highest by attribution changed the output 4.1 times more than removing three random rules, but it exceeded the median random removal in only 83.5% of non-degenerate rows (criterion: 90%). Of the 575 positive-control pairs, 552 increased strictly. We therefore do not claim validated rule-level explanations: the rule trace matches the inference exactly and the direction of the response is verified, but the attribution of the output to individual rules is faithful on average rather than uniformly. The degenerate rows, the post hoc fold breakdown and the paired-week contrast are in Supplementary note S3 of Additional file 1.

4.7 Attribution and monotonicity checks (E8)

We computed exact interventional Shapley values Shapley (1953) for the seven inputs of the 25 saved models (5,400 model–row pairs; a background of 60 random labelled rows, excluding P0 and P0-IF and identical for all models; the efficiency property holds to 2×10^{-14}). Exploitability (x_2) and the KEV indicator (x_7), both ingredients of the label, carry 78.9% of the mean absolute attribution; the three environment inputs (x_4–x_6) carry 3.3% (Table 6). The ranking is stable across models (Kendall $W = 0.79$; x_2 is the top input in 21 of 25 models), and in the 20% of rows with the highest label x_7 alone carries 63.0%. This quantifies the overlap between label and inputs discussed in Section 5: the model largely forecasts exploitability signals it can already see, and the label contains no environment component by construction.

Table 6 Exact Shapley attribution and within-row monotonicity checks for the five-seed ensemble (E8; descriptive and post hoc). "Falls" is the share of steps, over the range observed in the data, in which the score drops by more than 0.05 points when one input is raised and the others are held fixed. Share: mean absolute attribution; Max drop and Range: largest fall and mean end-to-end change of the score in points.

Input	Share (%)	Falls (%)	Max drop	Range
x_1 CVSS	10.9	1.7	2.46	1.99
x_2 Exploitability	46.4	1.2	0.21	6.53
x_3 Patch Risk	6.8	24.8	2.28	1.11
x_4 MFA Risk	1.1	29.1	0.46	0.09
x_5 Kubernetes Exposure	1.3	34.8	0.60	0.14
x_6 Asset Criticality	0.9	10.5	0.40	0.13
x_7 Threat Intelligence (KEV)	32.5	1.2	0.33	6.45

Non-negative rule slopes do not guarantee a monotone score, because rule weights are normalised and intercepts are free. We therefore varied one input at a time over 11 points spanning the observed range (observed values for discrete inputs, quantiles from 1% to 99% otherwise) for the five-seed ensemble that is reported in Section 4. For the dominant drivers the share of falling steps is small (x_1 1.7%, x_2 1.2%, x_7 1.2%), whereas for patch risk and the environment inputs it is 10–35%, although the environment inputs move the score by less than 0.15 points on average (Table 6).

Individual seeds are less well behaved than the ensemble: the largest single drop (11.6 points, CVSS) occurs in one seed on two assets in the last held-out block, after the EPSS model change. We therefore present monotonicity as an empirical property of the dominant inputs, not as a guarantee of the architecture.

4.8 Label-ingredient ablation and information-minimal baselines (E9–E11)

Both analyses were designed after the attribution results of Section 4.7 and are therefore post hoc. In E9 we retrained HANFIS with x_2 and x_7 replaced by the constant zero in training and test data, on the folds, seeds, embargo, validation split and hyperparameters of the confirmatory run, which we verified to reproduce the same 1,080 held-out rows. Skill scores are $\mathrm{SS} = 1 - \mathrm{MSE}_{\mathrm{model}}/\mathrm{MSE}_{\mathrm{reference}}$ with 95% bootstrap intervals clustered by image (2,000 resamples). Without the two label ingredients the five-seed ensemble has a mean squared error of 14.23, against 0.41 with them and 12.54 for a single CVSS score. Its skill over CVSS is -0.135 (interval $[-0.336, 0.046]$) and over a ridge model on the remaining five inputs -0.070 ($[-0.250, 0.096]$). That ridge model is itself no better than CVSS (-0.061, $[-0.130, 0.006]$). On this inventory the remaining inputs therefore carry no demonstrable information beyond CVSS, and essentially all of the model's skill comes from the two inputs that are also ingredients of the label.

In E10 we compared the confirmatory HANFIS predictions with the simplest functions of those inputs, trained on the same rows: a plug-in $100\,x_2$, ridge regression on (x_2, x_7), and gradient boosting on (x_2, x_7), on all seven inputs, and on the seven inputs plus the four-week lag of the label (Table 7). Within this group, HANFIS has a lower error than every alternative with an interval excluding zero, except gradient boosting restricted to x_2 and x_7 (skill 0.606, interval $[-0.079, 0.847]$). Adding the other inputs makes gradient boosting worse, not better, and models that receive the lagged label

as a feature do much worse than the lag itself, which we attribute, without having tested it, to distribution shift between training and test periods. Taken alone, these comparisons would suggest that HANFIS is a compact estimator of exposure from x_2 and x_7 that matches persistence without access to past labels; the next comparison qualifies this.

In E11 we asked whether a small formula suffices. Because the label is the window mean of a maximum over $\max(\text{EPSS}, \mathbb{1}[\text{KEV}])$ ρ, we fitted $\hat{y} = a + b\,\max(x_2,\, c\,x_7)$ on the training rows of each fold, choosing c on a grid and a, b by least squares, together with a ridge model with the $x_2 x_7$ interaction and monotone gradient boosting on (x_2, x_7). The grid selected $c = 0.10$ in all five folds, with b between 99.2 and 100.0 and $a < 0.07$, that is, almost exactly $100\,\max(x_2,\, 0.1\,x_7)$. This formula has a lower pooled error (0.27) than HANFIS (0.41) and than persistence (0.42). HANFIS is worse than the formula, with a skill of -0.534 (interval $[-1.438, -0.208]$), and worse than the interaction ridge (-0.529), while it is not distinguishable from monotone boosting (0.538, $[-0.005, 0.802]$). We had expected parity or an advantage for HANFIS; the observed disadvantage means that on this target the 176-parameter model adds no accuracy to what a four-parameter formula of current EPSS and KEV signals already provides. The gains over CVSS, linear models and a flat ANFIS reported in the confirmatory comparisons therefore show that the task is nearly determined by those two signals, not that the hierarchical structure helps.

4.9 Threat-intelligence ablation (E26)

H4 asks whether including the time-varying threat input improves prioritisation compared with an otherwise identical HANFIS without it. The registered protocols contain no test of H4, and E9 removes x_2 and x_7 together. After the forward evaluation we therefore ran the closest procedure (protocol `e26-protocol-2026-11-07`; post hoc, descriptive, unregistered, no forward data): HANFIS was retrained with only x_7, the

Table 7 Label-ingredient ablation (E9), information-minimal baselines (E10) and formula baselines (E11), 1,080 held-out rows. Skill score with 95% image-clustered bootstrap interval; MSE of the model and of the reference.

Comparison	Skill	95% interval	MSE
E9: HANFIS retrained without x_2 and x_7			
vs. CVSS only	−0.135	[−0.336, 0.046]	14.23 / 12.54
vs. ridge, remaining five inputs	−0.070	[−0.250, 0.096]	14.23 / 13.30
vs. HANFIS with all inputs	−33.8	[−56.5, −22.5]	14.23 / 0.41
Ridge, five inputs, vs. CVSS only	−0.061	[−0.130, 0.006]	13.30 / 12.54
E10: HANFIS with all inputs vs.			
Plug-in $100\,x_2$	0.859	[0.672, 0.931]	0.41 / 2.89
Ridge on (x_2, x_7)	0.693	[0.518, 0.817]	0.41 / 1.33
Ridge on lag-4 label, x_2, x_7	0.635	[0.399, 0.781]	0.41 / 1.12
Boosting on (x_2, x_7)	0.606	[−0.079, 0.847]	0.41 / 1.04
Boosting on seven inputs	0.659	[0.133, 0.860]	0.41 / 1.20
Boosting on lag-4 label and inputs	0.653	[0.104, 0.858]	0.41 / 1.18
Persistence (lag-4 label)	0.015	[−0.620, 0.243]	0.41 / 0.42
E11: HANFIS with all inputs vs.			
Max-form formula $a + b\max(x_2, c\,x_7)$	−0.534	[−1.438, −0.208]	0.41 / 0.27
Ridge on x_2, x_7, x_2x_7	−0.529	[−1.415, −0.206]	0.41 / 0.27
Monotone boosting on (x_2, x_7)	0.538	[−0.005, 0.802]	0.41 / 0.88

KEV indicator, set to zero, on the E4-bis folds, seeds, embargo and hyperparameters of E9. Without x_7 the pooled mean squared error rose from 0.41 to 3.16 (skill against the full model −6.738, 95% CI −13.958 to −2.899), in all five folds. The weekly ranking also fell: the Spearman correlation from 0.969 to 0.889 (difference −0.080, −0.089 to −0.072) and the precision at the top 20% from 0.973 to 0.716 (−0.258, −0.307 to −0.204). Without x_7, HANFIS still beat the CVSS-only baseline (skill +0.748, +0.605 to +0.856) but was far behind persistence (−6.625, −19.375 to −2.492). In the direction that H4 predicts, then, removing x_7 hurts. The reading is limited: x_7 is the KEV indicator, an ingredient of the label (Section 4.7), so the result measures the dependence of the model on a label ingredient, not the value of x_7 as an independent threat signal, and H4 as registered remains untested.

4.10 Trained rules and membership order (E27)

To document the rule base we printed the 34 rules of one model fixed beforehand (fold 0, first frozen seed) with the accessors of the model itself (Tables S11 to S13 of Additional file 1); the output of each panel rebuilt from these parameters equals the forward pass of the model on 1,000 fixed rows. Figure S4 of Additional file 1 shows the response of the five-seed ensemble of fold 0 over (x_2, x_7): it rises smoothly in both inputs, and the response to x_2 saturates inside the observed range. The Gaussian centres are free parameters, and nothing forces them to keep the order of their index. In the documented model the centres of x_2, v and e are not in index order. Reading the parameters of all 25 saved models (an extension beyond the protocol, descriptive, no training), the centres are in increasing order for x_1, x_3, x_4, x_5 and x_6 in all 25 models, for x_2 in 24 and for x_7 in 20, but for the intermediate variables in only 8 models for v and 17 for e. The names "low, medium, high" therefore cannot be read from the membership index of the intermediate variables in most trained models, although Section 3.2 chose three functions for them to keep that reading. The rule base is printable and its structure is fixed, but the meaning of its intermediate terms has to be recovered for each trained model.

4.11 Horizon sweep and feature impact by horizon (E14)

All preceding analyses use the four-week label window of the confirmatory protocol. To check whether the conclusions depend on that horizon, we rebuilt the dataset with label windows of 58 and 86 days (eight and twelve weekly snapshots after t). To avoid leakage we set the persistence lag and the embargo to the window length (8 and 12 weeks; 58 and 86 days). We then repeated the out-of-fold comparison of HANFIS, the max-form formula and persistence with the same folds, seeds and hyperparameters. Longer windows leave fewer labelled weeks (the last labelled week is 2026-07-27 and 2026-06-29), so the out-of-fold samples shrink to 889 and 697 rows, and mean squared

errors at different horizons are not comparable because the labels average different numbers of snapshots; only skill scores are compared. This analysis is post hoc.

The formula's advantage over persistence is stable (skill 0.36, 0.49 and 0.44 at four, eight and twelve weeks), whereas the gap between HANFIS and the formula closes: −0.534 (interval [−1.438, −0.208]) at four weeks, −0.048 ([−0.205, 0.032]) at eight and 0.039 ([−0.031, 0.110]) at twelve. HANFIS beats persistence from eight weeks on (0.467 and 0.462, intervals above zero). Because the longer horizons only contain older weeks (the EPSS model changed on 2026-06-15), horizon and period are confounded. Restricting all three horizons to the 408 asset-weeks present in each, the pattern remains: the skill of HANFIS against the formula changes by +0.862 ([+0.309, +2.973]) between four and twelve weeks, while the skill of the formula against persistence does not change (−0.016, [−0.096, +0.141]).

Removing exploitability and the KEV indicator again removes all skill at eight and twelve weeks (skill against CVSS alone −0.099 and −0.179, intervals including or touching zero; against HANFIS with all inputs −24.0 and −14.6). A ridge model on the remaining five inputs is not better than CVSS alone (−0.069 and −0.071). The relative gain of HANFIS at longer horizons does not come from the other inputs; Section 4.13 shows that it is largely due to weeks around the EPSS model change and that on unaffected rows HANFIS remains behind the formula. The evidence is suggestive, not conclusive: at twelve weeks HANFIS and the formula are not distinguishable, cross-fitted blends of the two do not improve on either alone, and the forward evaluation of Section 4.16 covers only the four-week horizon.

4.12 Prioritisation utility (E15)

Risk scores are used to rank assets, so we also compared the predictors on ranking within each week (post hoc; out-of-fold predictions at three horizons; weekly Spearman correlation and precision at the top 20%; Tables S5 and S6 of Additional file 1).

Table 8 Horizon sweep (E14, post hoc). Skill score with 95% image-clustered bootstrap interval. Panel A: all out-of-fold rows; B: the 408 asset-weeks present at all three horizons; C: HANFIS retrained without x_2 and x_7 against CVSS only.

Comparison	Skill	95% interval	Rows
A. All out-of-fold rows			
4 weeks: formula vs. persistence	0.357	[0.279, 0.407]	1080
4 weeks: HANFIS vs. persistence	0.015	[−0.620, 0.243]	1080
4 weeks: HANFIS vs. formula	−0.534	[−1.438, −0.208]	1080
8 weeks: formula vs. persistence	0.492	[0.269, 0.696]	889
8 weeks: HANFIS vs. persistence	0.467	[0.236, 0.651]	889
8 weeks: HANFIS vs. formula	−0.048	[−0.205, 0.032]	889
12 weeks: formula vs. persistence	0.440	[0.245, 0.597]	697
12 weeks: HANFIS vs. persistence	0.462	[0.277, 0.610]	697
12 weeks: HANFIS vs. formula	0.039	[−0.031, 0.110]	697
B. Same 408 asset-weeks at every horizon			
4 weeks: HANFIS vs. formula	−0.812	[−2.952, −0.264]	408
8 weeks: HANFIS vs. formula	−0.045	[−0.218, 0.038]	408
12 weeks: HANFIS vs. formula	0.050	[−0.060, 0.158]	408
Change 12 minus 4 weeks: HANFIS vs. formula	0.862	[0.309, 2.973]	408
Change 12 minus 4 weeks: formula vs. persistence	−0.016	[−0.096, 0.141]	408
C. HANFIS without x_2, x_7 vs. CVSS only			
4 weeks	−0.135	[−0.336, 0.046]	1080
8 weeks	−0.099	[−0.289, 0.069]	889
12 weeks	−0.179	[−0.406, 0.007]	697

All predictors rank assets almost identically. At four weeks the weekly Spearman correlation is 0.99 for the formula and for persistence and 0.97 for HANFIS, and the precision at the top 20% is 0.97 for all three, so the lower squared error of the formula does not translate into a practical gain in prioritisation. At longer horizons the differences stay small (Supplementary note S4 of Additional file 1), and with eighteen paired comparisons and no correction for multiplicity the intervals are descriptive.

4.13 Error anatomy and the EPSS model change (E16–E19)

The preceding sections compare pooled errors. To see where the differences come from, we analysed the four-week out-of-fold predictions by stratum, by time block and by

distance to the change of the EPSS model on 2026-06-15 (post hoc, with a bootstrap clustered by image).

Reactivity. The advantage of the formula over persistence is concentrated in rows in which an input changed during the previous four weeks (628 of 1,080 rows): its skill is 0.400 (interval 0.308 to 0.487) there and 0.002 (−0.069 to 0.037) where nothing changed (Table S7 of Additional file 1). HANFIS has skill 0.125 in changed rows and −0.900 (−4.171 to −0.206) in unchanged rows: it adds noise where persistence is already adequate. A gate that returns persistence in unchanged rows removes this deficit but leaves HANFIS no better than persistence (0.111, −0.367 to 0.292) and worse than the formula (−0.383, −0.998 to −0.131).

The model change. The largest errors are small shifts of the score in the weeks 2026-W21 to W24, whose four-week label windows straddle the EPSS model change: x_2 is computed with the old model while the label, and the label of the persistence forecast, use the new one. Rows within the label window of the change (190 of 1,080 at four weeks) hold 77% of the squared error of persistence and of the formula and 49% of that of HANFIS. On the remaining rows (Table 9) the formula keeps its point estimate of skill against persistence (0.348 against 0.357) but the interval now includes zero (−0.060 to 0.422); HANFIS is significantly worse than persistence (−1.228) and than the formula (−2.418). The forward evaluation (Section 4.16) is free of this change. At eight and twelve weeks the gap between HANFIS and the formula narrows (Section 4.11), but on rows unaffected by the change HANFIS stays behind the formula (−0.324 and −0.362, intervals including zero): the horizon sweep shows a shrinking deficit, not parity. The concentration of error, the leave-one-block-out estimates and the common-weeks analysis are in Supplementary note S5 of Additional file 1.

Table 9 Skill on all out-of-fold rows and on rows unaffected by the EPSS model change (E19, post hoc), with 95% image-bootstrap intervals from a separate run (they differ slightly from Table 8). Unaffected rows: 890 of 1,080 (four weeks), 536 of 889 (eight) and 342 of 697 (twelve).

Comparison	All rows	Unaffected rows
4 w: formula vs. persistence	0.357 [0.279, 0.407]	0.348 [−0.060, 0.422]
4 w: HANFIS vs. persistence	0.015 [−0.627, 0.252]	−1.228 [−12.804, −0.015]
4 w: HANFIS vs. formula	−0.534 [−1.383, −0.204]	−2.418 [−13.204, −0.709]
8 w: formula vs. persistence	0.492 [0.269, 0.696]	0.623 [−0.009, 0.699]
8 w: HANFIS vs. persistence	0.467 [0.241, 0.670]	0.501 [−1.631, 0.702]
8 w: HANFIS vs. formula	−0.048 [−0.200, 0.036]	−0.324 [−2.153, 0.052]
12 w: formula vs. persistence	0.440 [0.245, 0.597]	0.824 [0.270, 0.913]
12 w: HANFIS vs. persistence	0.462 [0.269, 0.614]	0.761 [−0.220, 0.888]
12 w: HANFIS vs. formula	0.039 [−0.034, 0.106]	−0.362 [−1.342, 0.018]

4.14 Calibration (E21)

We checked calibration after the registration of the forward evaluation had become public (post hoc, descriptive, no training); Table S8 of Additional file 1 gives the details. All three predictors lie close to the identity line. Persistence runs low on all rows (mean error −0.097, interval −0.196 to −0.013), almost entirely because of the weeks around the EPSS model change (on unaffected rows −0.015, −0.079 to 0.025). HANFIS departs most: its slope is 1.025 (1.008 to 1.044) on all rows and 1.031 (1.014 to 1.053) on unaffected rows, where its intercept is −0.193 (−0.300 to −0.085). These departures are small next to the label scale, and a bootstrap clustered by week (E22) gives narrower, not wider, intervals for them. We read the miscalibration of HANFIS as small but consistent in slope and intercept where the model change does not interfere, and as descriptive.

A linear recalibration (E23, post hoc) does not close the gap between HANFIS and the formula: on all rows the skill of HANFIS against the formula is −0.534 (−1.383 to −0.204) before and −0.535 (−1.287 to −0.232) after cross-fitted recalibration, so the deficit is not explained by a linear miscalibration. Stacking (E24, post hoc) finds no information in HANFIS that a straight line can use: the skill of the combination against the recalibrated formula is −0.001 (−0.006 to 0.004) on all rows. The full descriptions and all intervals are in Supplementary note S6 of Additional file 1.

4.15 Computational cost against flat ANFIS variants (E25)

The introduction and H2 promise a cost comparison that the confirmatory protocol did not contain. We therefore measured, after the forward evaluation and without using any of its data, the training time, peak memory, single-row inference latency and batch throughput of the frozen E4-bis procedures of HANFIS, FlatANFISMatched and the full flat ANFIS. The protocol was written and tagged before the measurement (`e25-protocol-2026-11-07`). The measurement ran once (five folds, three repetitions, the first frozen seed, one thread, CPU only, one mini PC under WSL 2) and is descriptive and unregistered. Table 10 sets it beside the accuracy results of Section 4.3; per-fold values are in Table S9 of Additional file 1.

Table 10 HANFIS against the two flat ANFIS variants. The accuracy rows come from the confirmatory E4-bis comparison (registered for FlatANFISMatched, reported without criterion for the full model; the fold-0 row is post hoc). The cost rows are the unregistered E25 measurement (median of 15 runs, first frozen seed, one machine). Flat ANFIS variants were not part of the forward evaluation.

	Flat ANFIS (full)	FlatANFISMatched	HANFIS
Rules / parameters	128 / 1,052	8 / 176	34 / 176
Skill of HANFIS against the model	0.253 (no criterion)	0.665 [0.269, 0.844] (H2/H3)	—
The same without fold 0 (post hoc)	0.084	0.183	—
Range of held-out predictions	not reported	−6.679 to 11.059	0.000 to 10.401
Training time, median (s)	7.30	0.95	8.51
Peak memory increase, median (MiB)	3.83	194.13	0.40
Single-row latency, median (μs)	95.6	49.1	398.2
Batch throughput, median (rows/s)	264,865	2,456,989	388,665

Relative to the full flat ANFIS, HANFIS has about six times fewer parameters (176 against 1,052), its peak memory increase during training is about a tenth (0.40 against 3.83 MiB, lower in all five folds), and its batch throughput is higher (388,665 against 264,865 rows per second, higher in all five folds). It was not faster to train (median 8.51 against 7.30 s, lower in two of five folds), and its single-row latency was about four times higher (398 against 96 μs). FlatANFISMatched has the same parameter budget but trained about nine times faster (0.95 s) and answered about eight times faster per

row (49 μs); its memory increase was far larger and grew with the number of training rows (5.5 to 663 MiB from fold 0 to fold 4), probably because of its cluster-radius search, which we did not test. The hierarchy therefore lowers parameters and peak training memory and raises batch throughput relative to the full flat model, but it does not reduce training time or single-row latency, and the part of H2 that promised those reductions is not supported.

The measurement has limits. It uses one seed. HANFIS training time includes three restarts, whereas the full flat model is trained once with a different patience, so early stopping differs between models. Latency includes the conversion from NumPy to PyTorch and three unoptimised panel passes, a property of this implementation. Memory is a process-level peak with a resolution of about 1 MiB. The machine is not an edge device. Flat ANFIS variants were not part of the forward evaluation, so no forward comparison with them exists.

4.16 Prospective evaluation (E7): forward results

The design of Section 3.8 was executed once on 2026-11-02, and no stage was repeated. The evaluation covers 6 forward weeks (2026-08-31 to 2026-10-05) and 720 asset-weeks from 120 images (119 image groups). No row was dropped under the pre-registered insufficient-evidence rule, so every asset stayed in the analysis, including `cockroachdb`. The EPSS model version stayed at v2026.06.15 throughout.

Table 11 lists the registered decisions. In weeks without an EPSS model change, HANFIS was not better than persistence: the skill score was -3.757 (95% CI -9.177 to -1.594), and the upper bound lies below the margin of 0.05 (P7a supported: HANFIS not better). The fixed blend was worse than persistence (SS -0.807, 95% CI -2.507 to -0.160). The max-form formula beat persistence (SS 0.340, 95% CI 0.060 to 0.628; MSE 0.261 against 0.396; one-sided $p = 0.0005$, Holm threshold 0.0125). P7e is confirmed. HANFIS was not better than the formula (SS -6.208, 95% CI -13.677

to −3.188; MSE 1.884 for HANFIS against 0.261 for the formula). The upper bound lies below 0.05, so P7f is supported. The mean weekly Spearman correlation between predicted and realised change was 0.066 (lower 95% bound −0.074); the lower bound does not exceed zero, so P7d is not demonstrated. A bootstrap clustered by week (6 weeks) gives the same decision for P7e and P7f (SS 0.340, 95% CI 0.100 to 0.758 for P7e). The reported p values equal 1/2001, the smallest value that a bootstrap of 2,000 resamples can give.

Table 11 E7 forward results: skill score (pooled mean squared error, bootstrap clustered by image, 2,000 resamples) with 95% CI. P7a and P7b form one Holm family, P7e and P7f another; P7e and P7f are post hoc in origin.

Hypothesis	Comparison	SS	95% CI	Decision
P7a	HANFIS against persistence	−3.757	[−9.177, −1.594]	Supported (not better)
P7b	Blend against persistence	−0.807	[−2.507, −0.160]	Worse
P7e	Formula against persistence	0.340	[0.060, 0.628]	Confirmed
P7f	HANFIS against formula	−6.208	[−13.677, −3.188]	Supported (not better)

Against the out-of-fold results of Section 4.8, where the MSE was 0.27 for the formula, 0.42 for persistence and 0.41 for HANFIS, the forward MSE of the formula (0.261) and of persistence (0.396) stayed close, while that of HANFIS rose to 1.884.

P7e and P7f have a post hoc origin: they were added to the protocol before the freeze but derive from analyses of earlier data (E11 to E14). The protocol is tagged `e7-prereg-2026-10-12` (commit `dbed812`) and registered on the Open Science Framework (`https://osf.io/ahcvs/`; DOI `10.17605/OSF.IO/AHCVS`); the forward outputs are tagged `e7-results-2026-11-02`, and a Zenodo version DOI will be added. This is one forward window of six weeks and 120 images with shared weekly shocks, so the result does not extend to other EPSS model versions or to other image sets. Analyses after the freeze (E14c and E15 to E24) are post hoc and descriptive; none of them changes a registered decision.

5 Discussion

For a practitioner who needs to rank vulnerabilities by near-term exploitability exposure today, the evidence supports a simple choice: the max-form formula of the EPSS and KEV signals, with persistence as the comparator that any alternative must beat. HANFIS remains relevant only where an auditable rule structure is required and where an outcome that depends on the environment is available to test the environmental panel.

Three findings from the completed experiments (E1, E3, and the observability-overhead runs) shape how the confirmatory comparison in Section 4.3 should be read. First, the label is dominated by its own recent history. A 4-week persistence forecast reduces mean squared error relative to climatology by 97% on the 120-asset inventory (Table 3; 96% on the earlier pilot), so any claim that HANFIS "predicts risk" is a claim about improving on a strong autoregressive baseline, not about beating an uninformative one. This is why H1 was reframed against persistence rather than against Fuzzyfortify or CVSS alone (Section 1). Second, a single CVSS score has limited explanatory power for the realised 30-day exploitability exposure ($SS_{clim} = 0.222$ on the 120-asset inventory, against about 0 on the earlier pilot). Severity and realised exploitation are different quantities, and conflating them is a plausible source of alert fatigue in practice; this is consistent with the research gap that C3 claims to address. Third, the E1 ablation shows that a monotonicity constraint intended to keep the model auditable (higher risk factors must not lower the score) has a family-dependent accuracy cost on data containing thresholds. Choosing Gaussian membership functions with monotonicity active therefore trades some accuracy for a response that, for the dominant risk factors, rarely falls as they rise (Section 4.7); it was a deliberate choice.

The post hoc analyses of Sections 4.7 and 4.8 limit how far these findings generalise. Exact Shapley attribution places 78.9% of the model's attribution on exploitability and the KEV indicator, which are ingredients of the realised label, and only 3.3%

on the three environmental inputs. Without those two inputs the model shows no demonstrable skill over a single CVSS score, and it is not demonstrably better than gradient boosting restricted to them. What the data support is therefore narrower. A 34-rule hierarchical model estimates near-term exploitability exposure from current EPSS and KEV signals about as accurately as a persistence forecast that needs past labels (the two are not distinguishable), and more accurately than generic linear and boosted models. It is not more accurate than a four-parameter formula of the same two signals (Section 4.8); this formula, together with the interaction ridge, has the lowest error of all predictors we compared at the four-week horizon. At eight and twelve weeks the gap to the formula narrows, but only through weeks around the EPSS model change (Sections 4.11 and 4.13). The ranking of assets by predicted risk is almost the same for all predictors at four weeks (Section 4.12). The data do not support the claim that the vulnerability–environment decomposition, or the environmental inputs, improve prediction. Because the target is built from EPSS, KEV and reachability alone and does not depend on the three environmental inputs, this inventory is not designed to show a benefit of those inputs even if one exists; testing that benefit requires an outcome that depends on the environment, such as incident or impact data. Forward weeks confirm the earlier finding: a max-form formula in the exploitability and threat-intelligence inputs matches or beats the trained model, and HANFIS adds structure and a readable rule base, not accuracy (Section 4.16). In those weeks HANFIS was not better than persistence (P7a) and its blend with persistence was worse than persistence alone (P7b). We did not test why the trained model transferred worse than the formula. Its rule base can be printed (Tables S11 to S13 of Additional file 1), but the linguistic labels of its intermediate variables cannot be read from the membership index in most trained models (Section 4.10), so the readability of the rule base is a property of its structure, not of its terms.

The instrumentation-overhead results (Section 4.1) support a second, narrower claim: Kubernetes observability telemetry can be treated as a fixed resource cost rather than folded into the risk model as a confound. Node-level CPU increases by 99.4 mCPU under monitoring, while the monitoring namespace itself accounts for only 39.8 mCPU, so pod-level accounting alone understates the platform cost by a factor of about 2.5. Workload latency nonetheless shows no detectable change (95% confidence intervals within ± 0.27 ms). The cost of observing the system is thus separated from any change in the system being observed, which is the premise for any later use of runtime telemetry to derive the Kubernetes-exposure input. Whether the exposure input improves prediction is a separate question, and on the present inventory the environmental inputs add no demonstrable predictive information (Section 4.8).

5.1 Limitations

Two limits of construct validity come first. First, the label and two of the inputs share ingredients: x_2 (EPSS $\times\, \rho$) and x_7 (the KEV indicator) are the values at t of quantities that the label aggregates over $t{+}7, \ldots, t{+}28$, and runtime reachability ρ enters both. The task is therefore to forecast the near future of exploitability signals that the model can already see, not to predict independent compromise outcomes, for which no ground truth exists here. The persistence baseline, built in the same way, is the relevant bar, and the comparison with a single CVSS score should be read in that light. The exact Shapley attribution (Section 4.7) and the ablation without these two inputs (Section 4.8) show how heavily the model's skill rests on them. Second, the dynamic character of the score comes from the time-varying EPSS and KEV feeds and from runtime reachability. Runtime telemetry (CPU, memory, latency) is used only to quantify observability overhead and is not a model input, so "dynamic" must not be read as telemetry-driven scoring. The rule-level attribution is faithful on average but not uniformly (Section 4.6), and we do not claim validated explanations.

All analyses labelled post hoc were designed after the confirmatory results and run on the same data; they are descriptive, carry no multiplicity correction, and none of them changes a registered decision. The cost measurement (E25), the x_7 ablation (E26) and the rule documentation (E27) were run after the forward evaluation, use no forward data, and are unregistered.

The inventory consists of public container images and a small number of testbed assets built from a public base image, and the label reflects exploitation signals of public vulnerabilities (EPSS, KEV), not incidents in a specific organisation; generalisation to other workloads or to incident-based outcomes is untested.

Three data-related caveats are detailed in Supplementary note S7 of Additional file 1. On the 120-asset inventory (6,357 labelled rows; Table 3), a linear ridge model over all seven inputs underperforms persistence on every one of the five grouped-temporal folds ($SS_{pers} = -2.293$ pooled, raw scale), and we did not test why. The EPSS scoring model changed version (`v2025.03.14` to `v2026.06.15`) inside the sealed 57-week window, a threat to label comparability; a post hoc re-analysis restricted to weeks whose labels all pre-date the change leaves the confirmatory conclusions unchanged (Section 4.5). The data-sufficiency gate (at least ten observations per parameter) is met by the pooled sample (6,357 labelled rows against 176 parameters) but not by the two earliest held-out folds (2.2 and 7.1 observations per parameter), which are also those in which HANFIS is furthest behind persistence; The H2/H3 pooled margin is dominated by the first fold (Section 4.5).

CVEs lacking a Trivy-reported publication date are dated by their first appearance in a sealed EPSS snapshot (at most 7 days late), and a single CVE whose Trivy status lies outside the accepted set makes the whole asset-week unusable; this affected one of the 120 images for seven consecutive weeks from 2026-08-10, and such rows are removed. Labels whose four-week window straddles the EPSS model change of 2026-06-15 (and the persistence labels derived from them) mix two score definitions; these

rows (18% of the out-of-fold rows) hold 77% of the squared error of persistence and of the formula, and the forward evaluation avoids them. The label aggregation rule, its numeric scale and the definition of Patch Risk were closed and pre-registered as amendment A-13 before the confirmatory comparison in Table 4 was run.

The testbed is a single two-node WSL 2/Docker Desktop Kubernetes cluster; external validity to production multi-node clusters and other CNI/storage stacks is untested. Ten paired observability runs, while showing large and consistent effects, still provide limited power for null-hypothesis testing on the latency measure specifically, which is why the reported confidence intervals rather than a significance threshold carry the interpretive weight in Section 4.1.

6 Conclusion and Future Work

This paper asked whether a domain-informed hierarchical ANFIS earns its complexity when scoring the near-term exploitability exposure of container images. On this target it does not. HANFIS reduces a 128-rule, 1,052-parameter flat Sugeno system over seven cloud-native risk inputs to 34 rules and 176 parameters (Section 3.2) by separating vulnerability and environmental reasoning into two first-tier panels before an integration tier combines them with dynamic threat intelligence. Under a protocol frozen and dated before training began (Table 4), it confirms a substantial reduction in estimation error against a single CVSS score (H1a), a linear model over the same seven inputs (H1c), and a parameter-matched flat ANFIS initialised by subtractive clustering (H2/H3). It is not distinguishable from a causal 4-week persistence forecast (H1b), which the study's own mandatory baselines (Section 4.3) show to be a far harder bar than static expert scoring: persistence alone removes 97% of the error relative to climatology. The pooled margin over the matched model is dominated by the first block, where training is thinnest, so it is read as suggestive of a structural advantage of the vulnerability/environmental/threat decomposition and is not evidence for

one (Section 4.5). Against the flat ANFIS variants, HANFIS lowers parameters and peak training memory but not training time or single-row latency (Section 4.15).

Post hoc analyses qualify this further. The model's skill rests on exploitability and KEV (78.9% of the exact Shapley attribution; without them it has no skill over CVSS), and a four-parameter formula of those two inputs has a lower error than HANFIS (Sections 4.7 and 4.8). On this target HANFIS therefore provides a compact rule base with a documented response surface, while its four-week accuracy stays below that of a simple formula. Its intermediate variables do not keep a stable linguistic meaning across trained models (Section 4.10). The pre-registered forward evaluation (Section 4.16) points the same way: the accuracy of HANFIS on this task is bounded by a simple formula, and what the model adds is a printable structure. At eight and twelve weeks the gap narrows but does not close on rows unaffected by the EPSS model change, and the other inputs do not contribute (Sections 4.11 and 4.13). Whether environmental context improves forecasts remains open. Two further results are documented: the sealed, hash-verified real-world data pipeline (120 assets and 6,357 labelled asset-weeks, Section 3.5) and an ablation on two synthetic scenarios indicating that the monotonicity constraint has a membership-family-dependent accuracy cost, met in practice only for the dominant drivers (Sections 4.5 and 4.7).

Three points follow for practice and for later work. For ranking near-term exploitability exposure from EPSS and KEV signals, the max-form formula, with persistence as the bar that any alternative must beat, is the choice the evidence supports (Section 5). A hierarchical neuro-fuzzy model is worth building only where an auditable rule structure is required, or where an outcome that depends on the environment, such as incident or impact data, is available to test the environmental panel; the present target cannot show that benefit even if it exists. The result concerns one architecture, one label and one six-week forward window, and it does not extend to other EPSS model versions, other image sets or other neuro-fuzzy designs.

H1b against persistence remains unconfirmed, and the forward weeks did not support it either (P7a). The bottleneck-width sweep (Section 4.4) found that widening the scalar inter-tier bottleneck from k=1 to k=2 does not close the gap to persistence; the wider model is significantly worse than persistence. Re-scanning every image with a single frozen vulnerability database, so that no image's CVE surface differs merely because it was scanned on a different day, was done once for the forward evaluation (Section 4.16); the earlier 57-week window still rests on scans from different days. The policy-translation layer (not a contribution of this paper) and edge deployment (originally hypothesis H5, now future work because the present testbed runs HANFIS cluster-side rather than on its edge node) remain designed but unevaluated. Both are left for a subsequent study.

List of abbreviations

AI Artificial intelligence

ANFIS Adaptive neuro-fuzzy inference system

ANN Artificial neural network

CI Confidence interval

CISA Cybersecurity and Infrastructure Security Agency

CPU Central processing unit

CVE Common Vulnerabilities and Exposures

CVSS Common Vulnerability Scoring System

DOI Digital Object Identifier

EPSS Exploit Prediction Scoring System

FIS Fuzzy inference system

HANFIS Hierarchical adaptive neuro-fuzzy inference system

HTTP Hypertext Transfer Protocol

KEV Known Exploited Vulnerabilities

LPPM Institute for Research and Community Service

MFA Multi-factor authentication

MSE Mean squared error

OSF Open Science Framework

RMSE Root mean squared error

SHA Secure Hash Algorithm

SS Skill score

WSL Windows Subsystem for Linux

Declarations

- **Funding.** This work is supported by the Penelitian Kepakaran research grant, Institute for Research and Community Service (LPPM), Universitas Andalas, Indonesia. The funder had no role in the design of the study, in the collection, analysis or interpretation of data, or in writing the manuscript.

- **Competing interests.** The authors declare no competing interests.
- **Ethics approval and consent to participate.** Not applicable: this study involves no human participants, human data, or animal subjects. All measurements were collected from an isolated Kubernetes research testbed operated by the authors.
- **Consent for publication.** Not applicable.
- **Availability of data and materials.** The sealed EPSS/KEV evidence manifests, container-image vulnerability scan bundles, derived asset-week tables, frozen model files and analysis scripts are archived at Zenodo (DOI `10.5281/zenodo.23357118`); the forward-evaluation protocol is registered at the Open Science Framework (DOI `10.17605/OSF.IO/AHCVS`). Redistribution of the underlying EPSS and CISA KEV feeds follows their licensing terms. The raw telemetry outputs of the ten paired observability-overhead runs (Runs 7–16) are not included in version 1.0 of that record and will be deposited with a later version.

- **Code availability.** The HANFIS implementation, data pipeline and experiment scripts are archived at Zenodo (DOI `10.5281/zenodo.23357118`) and developed at `udahafeez7/HANFIS-RESEARCH` (private during review).
- **Authors' contributions.** M.H.H.: conceptualisation, methodology, software, experimentation, writing – original draft. D.D.: supervision, contribution to the neural network component, methodology review, writing – review and editing. B.R.: supervision, contribution to the design of the experimental platform (cluster and telemetry network), methodology review, writing – review and editing.
- **Use of AI tools.** During the preparation of this work the authors used an AI assistant (Claude, Anthropic; Claude Sonnet 5.5, Claude Opus 5.5 and Claude Fable 5.1 models, 2026) to help write and test code, to draft the two unregistered protocols, to run scripts at the authors' direction, to draft and revise manuscript text, and to check references. The authors decided every protocol and claim, verified all code, results and references, reviewed and edited all text, and take full responsibility for the content of the publication. The assistant is not an author.
- **Acknowledgements.** The authors thank the Institute for Research and Community Service (LPPM), Universitas Andalas, for supporting this work.

References

Allodi L, Massacci F (2014) Comparing vulnerability severity and exploits using case-control studies. ACM Transactions on Information and System Security 17(1):1–20. https://doi.org/10.1145/2630069

Aqua Security (2026) Trivy. https://github.com/aquasecurity/trivy, version 0.74.0; vulnerability database of 2 October 2026

Brown M, Bossley KM, Mills DJ, et al (1995) High dimensional neurofuzzy systems: overcoming the curse of dimensionality. In: Proceedings of 1995 IEEE International

Conference on Fuzzy Systems, vol 4. IEEE, pp 2139–2146, https://doi.org/10.1109/FUZZY.1995.409976

Chandramouli R (2019) Security strategies for microservices-based application systems. Tech. Rep. NIST Special Publication 800-204, National Institute of Standards and Technology, https://doi.org/10.6028/NIST.SP.800-204

Chandramouli R, Butcher Z (2023) A zero trust architecture model for access control in cloud-native applications in multi-location environments. Tech. Rep. NIST Special Publication 800-207A, National Institute of Standards and Technology, https://doi.org/10.6028/NIST.SP.800-207A

Chiu SL (1994) Fuzzy model identification based on cluster estimation. Journal of Intelligent & Fuzzy Systems 2(3):267–278. https://doi.org/10.3233/IFS-1994-2306

Cybersecurity and Infrastructure Security Agency (2026) Known exploited vulnerabilities catalog. https://www.cisa.gov/known-exploited-vulnerabilities-catalog, weekly snapshots, 25 August 2025 to 21 September 2026

Efron B (1979) Bootstrap methods: Another look at the jackknife. The Annals of Statistics 7(1):1–26. https://doi.org/10.1214/aos/1176344552

FIRST.org, Inc. (2023) Common vulnerability scoring system version 4.0: Specification document. https://www.first.org/cvss/v4.0/specification-document, accessed 6 October 2026

Hafiz Hersyah M, Hossain MD, Taenaka Y, et al (2025) Fuzzyfortify: a multi-attribute risk assessment for multi-factor authentication and cloud container orchestration. Frontiers in Computer Science 7:1557918. https://doi.org/10.3389/fcomp.2025.1557918

Holm S (1979) A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics 6(2):65–70

Jabeen S, Baig M, Awais MM (2023) ANFIS learning using expectation maximization based Gaussian mixture model and multilayer perceptron learning. Applied Soft Computing 149:110958. https://doi.org/10.1016/j.asoc.2023.110958

Jacobs J, Romanosky S, Adjerid I, et al (2020) Improving vulnerability remediation through better exploit prediction. Journal of Cybersecurity 6(1):tyaa015. https://doi.org/10.1093/cybsec/tyaa015

Jacobs J, Romanosky S, Edwards B, et al (2021) Exploit prediction scoring system (EPSS). Digital Threats: Research and Practice 2(3):1–17. https://doi.org/10.1145/3436242

Jacobs J, Romanosky S, Suciu O, et al (2023) Enhancing vulnerability prioritization: Data-driven exploit predictions with community-driven insights. In: 2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). IEEE, pp 194–206, https://doi.org/10.1109/EuroSPW59978.2023.00027

Jang JSR (1993) ANFIS: Adaptive-network-based fuzzy inference system. IEEE Transactions on Systems, Man, and Cybernetics 23(3):665–685. https://doi.org/10.1109/21.256541

Joint Task Force Transformation Initiative (2012) Guide for conducting risk assessments. Tech. Rep. NIST Special Publication 800-30 Rev. 1, National Institute of Standards and Technology, https://doi.org/10.6028/NIST.SP.800-30r1

Juston MFR, Dekhterman SR, Norris WR, et al (2024) Hierarchical rule-base reduction-based ANFIS with online optimization through DDPG. IEEE Transactions on Fuzzy Systems 32(11):6350–6362. https://doi.org/10.1109/TFUZZ.2024.

3449147

Kim Y, Park C, Hong D (2025) A robust framework for comprehensive container image vulnerability assessment. IEEE Access 13:35837–35847. https://doi.org/10.1109/ACCESS.2025.3545124

Koscinski V, Nelson M, Okutan A, et al (2025) Conflicting scores, confusing signals: An empirical study of vulnerability scoring systems. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. ACM, pp 1904–1918, https://doi.org/10.1145/3719027.3765210

Lee ML, Chung HY, Yu FM (2003) Modeling of hierarchical fuzzy systems. Fuzzy Sets and Systems 138(2):343–361. https://doi.org/10.1016/S0165-0114(02)00517-1

Mills A, White J, Legg P (2023) Longitudinal risk-based security assessment of docker software container images. Computers & Security 135:103478. https://doi.org/10.1016/j.cose.2023.103478

Minna F, Blaise A, Rebecchi F, et al (2021) Understanding the security implications of Kubernetes networking. IEEE Security & Privacy 19(5):46–56. https://doi.org/10.1109/MSEC.2021.3094726

Ojha V, Abraham A, Snášel V (2019) Heuristic design of fuzzy inference systems: A review of three decades of research. Engineering Applications of Artificial Intelligence 85:845–864. https://doi.org/10.1016/j.engappai.2019.08.010

Parthiban S, Krishna S (2026) Hierarchical ANFIS-SMC framework for eco-driving-based energy optimization of electric vehicles. IEEE Access 14:71845–71855. https://doi.org/10.1109/ACCESS.2026.3692296

Rahman A, Shamim SI, Bose DB, et al (2023) Security misconfigurations in open source Kubernetes manifests: An empirical study. ACM Transactions on Software Engineering and Methodology 32(4):1–36. https://doi.org/10.1145/3579639

Raju GVS, Zhou J, Kisner RA (1991) Hierarchical fuzzy control. International Journal of Control 54(5):1201–1216. https://doi.org/10.1080/00207179108934205

Rose S, Borchert O, Mitchell S, et al (2020) Zero trust architecture. Tech. Rep. NIST Special Publication 800-207, National Institute of Standards and Technology, https://doi.org/10.6028/NIST.SP.800-207

Shapley LS (1953) A value for n-person games. In: Contributions to the Theory of Games (AM-28), Volume II. Princeton University Press, p 307–318, https://doi.org/10.1515/9781400881970-018

Shimizu N, Hashimoto M (2026) Vulnerability management chaining: An integrated framework for efficient cybersecurity risk prioritization. IEEE Access 14:31407–31424. https://doi.org/10.1109/ACCESS.2026.3665768

Shu R, Gu X, Enck W (2017) A study of security vulnerabilities on Docker Hub. In: Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy. ACM, pp 269–280, https://doi.org/10.1145/3029806.3029832

Soozanchi-K. Z, Akbarzadeh-T. MR, Yaghoobi M, et al (2010) Speaker verification system using a hierarchical adaptive network-based fuzzy inference systems (HANFIS). In: Artificial Intelligence in Theory and Practice III. IFIP Advances in Information and Communication Technology, Springer Berlin Heidelberg, p 233–237, https://doi.org/10.1007/978-3-642-15286-3_23

Souppaya M, Morello J, Scarfone K (2017) Application container security guide. Tech. Rep. NIST Special Publication 800-190, National Institute of Standards and

Technology, https://doi.org/10.6028/NIST.SP.800-190

Spring J, Hatleback E, Householder A, et al (2021) Time to change the CVSS? IEEE Security & Privacy 19(2):74–78. https://doi.org/10.1109/MSEC.2020.3044475

The Kubernetes Authors (2026) Pod lifecycle. https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/, accessed 6 October 2026

The Prometheus Authors (2026) Prometheus: Overview. https://prometheus.io/docs/introduction/overview/, accessed 14 October 2026

Torra V (2002) A review of the construction of hierarchical fuzzy systems. International Journal of Intelligent Systems 17(5):531–543. https://doi.org/10.1002/int.10036

Wang LX (1998) Universal approximation by hierarchical fuzzy systems. Fuzzy Sets and Systems 93(2):223–230. https://doi.org/10.1016/S0165-0114(96)00197-2

Wu HM, Zaman MQ (2025) Enhanced hierarchical fuzzy formation control with fuzzy collision avoidance behavior for multiple Mecanum wheeled mobile robots. Robotics and Autonomous Systems 194:105124. https://doi.org/10.1016/j.robot.2025.105124

Zeng XJ, Keane JA (2005) Approximation capabilities of hierarchical fuzzy systems. IEEE Transactions on Fuzzy Systems 13(5):659–672. https://doi.org/10.1109/TFUZZ.2005.856559

Zerouali A, Mens T, Robles G, et al (2019) On the relation between outdated Docker containers, severity vulnerabilities, and bugs. In: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE, pp 491–501, https://doi.org/10.1109/SANER.2019.8668013

Zhang Y, Wang G, Zhou T, et al (2024) Takagi-sugeno-kang fuzzy system fusion: A survey at hierarchical, wide and stacked levels. Information Fusion 101:101977. https://doi.org/10.1016/j.inffus.2023.101977